DBMS (2BTCO302) — Previous Year Question Papers with Model Answers

Dr. Subhash University · B.Tech CSE (AI/ML) · Semester 3 · Winter 2024 + Winter 2025 Source folder analysed: `dbms` (Google Drive) — 16 PDFs → 2 actual exam papers, 4 assignment sheets, 10 unit-wise notes.

Exam format (identical both years): **5 questions, all compulsory, internal OR choice in Q2–Q5, 100 marks, 3 hours.** Q.1 = 10 × 2-mark short answers **[20]** · Q.2 = 4+6+10 **[20]** · Q.3 = 4+6+10 **[20]** · Q.4 = 4+6+10 **[20]** · Q.5 = 4+6+10 **[20]**

---

1. What actually repeats (the paper-setter's fingerprints)

Comparing Winter 2024, Winter 2025 and Assignments 1–4 word-by-word, the pattern is unmistakable: the same topic bank is recycled, only the wording changes.

Topic	W2024	W2025	Assignments	Marks	Verdict
B-Tree construction	Q5(C) OR, order 5	Q5(C) OR, order 4	A3 Q7, order 4	**10**	Appears every single year. Guaranteed 10 marks.
Relational algebra – all operations	Q2(C)	Q2(C)	A2 Q1, Q2	**10**	Both years, same wording
ACID properties	Q3(C)	Q3(C)	A3 Q3	**10**	Verbatim identical in all three
Normalization + convert to 2NF/3NF	Q4(C)	Q5(A)+Q5(C)	A2 Q2, Q5	**10**	Both years
ER model / components of ER diagram	Q3(C) OR	Q3(C) OR	A1 Q7	**10**	Both years
DDL & DML commands with syntax	Q2(C) OR	Q2(C) OR	A1 Q1	**10**	Both years, the alternate of relational algebra
Types of join with example	Q3(A) OR	Q3(A) OR	A2 Q6	4	Both years, same position
Armstrong's axioms	Q3(B)	Q3(B)	A2 Q4	6	Verbatim identical
Sparse vs dense index	Q4(B)	Q5(B) OR	A3 Q1	6	Both years
GRANT / REVOKE	Q2(A)	Q5(A)	—	4	Both years
Network model	Q2(B)	Q3(A)	A1 Q6	4–6	Both years
PL/SQL block – max of 3 numbers	Q5(B)	Q4(A)	A4 Q8	4–6	Both years
Data independence + types	Q1(A)	Q2(A)	A1 Q5	2–4	Both years
Data & information	Q1(H)	Q1(A)	—	2	Both years
Data abstraction + types	Q3(B) OR	Q1(D)	A1 Q2	2–6	Both years
CREATE table syntax	Q1(E)	Q1(F)	A4 Q6	2	Both years
Numeric functions	Q4(A) any 4	Q4(B) any 6	A4 Q7	4–6	Both years
Authentication / Authorization / DAC / MAC	Q1(F)	Q4(A) OR + Q1(I)	A3 Q5, Q6	4	Both years
Trigger (syntax + types)	Q4(C) OR	Q1(H)	A4 Q2	2–10	Both years
View + types	Q4(B) OR	—	A4 Q1	6	High probability
Cursor + types	—	Q4(C)	A4 Q5	10	New in 2025, now in assignments
MySQL vs Oracle	—	Q3(B) OR	—	6	New in 2025
TRC vs DRC	—	Q1(C)	A2 Q7	2	New in 2025
Hashing + types	Q5(C)	—	A3 Q2	10	Alternate of B-Tree
Timestamp / 2-Phase locking protocol	Q5(A)	—	A3 Q4	4	Alternate of B-Tree
DB Recovery + types	—	Q4(C)	—	10	New in 2025
Query optimization	—	Q1(J)	—	2	Unit 5, still untested in long form

Two things worth knowing before you study. The 10-mark slot in Q2 is always *Relational algebra OR DDL/DML*, and the 10-mark slot in Q5 is always *B-Tree OR hashing OR normalization-conversion OR cursor*.

That means roughly **40 of 100 marks** sit in just four topics: B-Tree, Relational Algebra, ACID/Transaction, and Normalization. Prioritise accordingly.

Second: the assignment questions are pulled from the same bank as the exam, and several are word-for-word identical (ACID, Armstrong, sparse/dense, B-Tree, network model). If a question is in an assignment and also in a past paper, it is the strongest possible candidate for your paper.

---

2. The 20-mark Q.1 bank — short answers (both years, all 20 items)

Q.1 is pure recall: 10 questions × 2 marks. Answer in 3–5 lines with one example or one table. These are the fastest 20 marks in the paper.

Winter 2024 — Q.1

(A) Define data independence. The ability to change the schema at one level of a DBMS without changing the schema or the programs at the next higher level. *Logical data independence* = change the conceptual schema (add tables/columns, rename) without rewriting external views or applications. *Physical data independence* = change storage structure, indexes or file organisation without affecting the conceptual or external level.

(B) Difference between DDL and DML.

DDL (Data Definition Language)	DML (Data Manipulation Language)
Defines and modifies the **structure/schema**	Inserts, updates, deletes **data in** the structure
CREATE, ALTER, DROP, TRUNCATE, RENAME	INSERT, UPDATE, DELETE (SELECT is DQL)
Changes are usually committed automatically	Changes are transaction-controlled (COMMIT/ROLLBACK)
Operates on the dictionary/catalog; no WHERE clause	Operates on tuples; uses WHERE for row selection
Result = table definition	Result = modified rows

(C) Advantages of normalization. Removes insertion, deletion and update anomalies; reduces data redundancy and therefore storage; improves data integrity through smaller dependency-preserving tables; gives a cleaner, more stable logical design; makes the schema easier to maintain and to extend. (State the mirror disadvantages too if asked: over-normalization creates many tables and expensive joins.)

(D) Left outer join vs right outer join. `LEFT OUTER JOIN` returns **all rows of the left table** plus matched rows of the right; unmatched right-side columns become NULL. `RIGHT OUTER JOIN` returns **all rows of the right table** plus matched left rows; unmatched left columns become NULL. Direction of the "keep everything" table is the only difference.

(E) Syntax of CREATE command.

```
CREATE TABLE student (
    roll_no   INT         PRIMARY KEY,
    name      VARCHAR(30) NOT NULL,
    email     VARCHAR(50) UNIQUE,
    cgpa      DECIMAL(4,2) DEFAULT 0,
    dept_id   INT,
    CONSTRAINT fk_dept FOREIGN KEY (dept_id) REFERENCES department(dept_id)
);
```

Also valid forms: `CREATE DATABASE`, `CREATE VIEW v AS SELECT ...`, `CREATE INDEX i ON t(col)`, `CREATE TABLE new LIKE old`.

(F) Define Authentication. The process of verifying **who a user claims to be** before granting access to the database, using one or more factors — something you know (password, PIN), something you have (OTP, token, smart card, certificate), something you are (biometric). It is the gate that precedes *authorization*, which decides *what* the verified identity may do.

(G) Types of index. Primary index, secondary (alternate) index, clustering index, non-clustering index, dense index, sparse index, unique index, composite/concatenated index, multi-level index, hash index, B-Tree/B+Tree index, bitmap index.

(H) Define data and information. *Data* = raw, unprocessed facts and figures without context (e.g. `78`, `21B01234`, `"Bhavik"`). *Information* = data that has been processed, organised or put in context so that it is meaningful and useful for decision-making (e.g. "Bhavik scored 78/100 in DBMS, which is above the class average"). Data is the input; information is the output.

(I) Define subquery. A query nested inside another query (`SELECT`, `INSERT`, `UPDATE`, `DELETE`), whose result is used by the outer query. It executes first and returns a value, list or table.

```
SELECT name, cgpa FROM student
WHERE cgpa > (SELECT AVG(cgpa) FROM student WHERE dept_id = 3);
```

Types: single-row (scalar), multi-row with `IN/ANY/ALL`, correlated (re-executes per outer row), and in `FROM` (inline view/derived table).

(J) Two differences between SQL and PL/SQL. SQL is a **declarative, statement-by-statement** query language — one statement executes at a time and it cannot store control flow. PL/SQL is Oracle's **procedural extension** of SQL — it groups statements in blocks (`DECLARE…BEGIN…EXCEPTION…END`), supports variables, loops, conditions, cursors, procedures and exception handling, and sends the whole block to the engine in one context switch, so it is faster for procedural work.

Winter 2025 — Q.1

(A) What is data and information? Same as 2024 (H) above — expect it every year.

(B) Types of data models. A data model is a collection of conceptual tools for describing data, relationships, semantics and constraints. 1. **Entity-Relationship (ER) model** — entities, attributes, relationships; used for conceptual design. 2. **Relational model** — data as tables (relations) of rows; keys and constraints; most widely used. 3. **Hierarchical model** — tree/parent-child (IMS); one parent per child. 4. **Network model** — graph with owner–member sets, many-to-many via pointers (CODASYL). 5. **Object-Oriented model** — objects, classes, inheritance, encapsulation, complex data types. 6. **Semi-structured model** — self-describing, irregular structure: XML, JSON, document stores.

(C) Difference between TRC and DRC.

Tuple Relational Calculus (TRC)	Domain Relational Calculus (DRC)
Variable ranges over **tuples/rows**	Variable ranges over **domains (attribute values)**
Form: `{ t | P(t) }`	Form: $\{ \langle x_1, \ldots, x_n \rangle \mid P(x_1, \ldots, x_n) \}$
Written closer to SQL with tuple vars	Written as attribute-value patterns
Both are **non-procedural/declarative**, both restricted to *safe* expressions, and together they are expressively equivalent to relational algebra (Codd's theorem)	

Example — students in CSE: TRC `{ s \| Student(s) ∧ s.dept = 'CSE' }`; DRC `{ <r,n,d> \| Student(r,n,d) ∧ d = 'CSE' }`.

(D) What is Data Abstraction? Hiding internal details of data storage and management from users, exposing only what is relevant. Three levels: **internal/physical** (how data is actually stored — blocks, indexes), **conceptual/logical** (what data exists and relationships — tables, constraints), **external/view** (per-user subset of the database). Its benefit is simplicity for users and flexibility for developers — the basis of data independence.

(E) Define functional dependency. A constraint between two attribute sets written `X → Y` on relation R: for every pair of tuples, if they agree on X they must also agree on Y. X is the *determinant*, Y the *dependent*. It expresses that Y is uniquely determined by X. Types: trivial (`X → Y` where Y ⊆ X), non-trivial, fully/partially dependent, transitive, and multi-valued (X →→ Y).

(F) Syntax of table creation. Same as 2024 (E) — write the full form with at least PK, FK, NOT NULL, UNIQUE, CHECK and DEFAULT.

(G) Primary key vs foreign key.

Primary Key	Foreign Key
Uniquely identifies each row of **its own** table	Refers to the primary key of **another** (or same) table
Must be UNIQUE and **NOT NULL**; only one per table	May contain NULLs and duplicates; many per table allowed
Establishes entity integrity	Establishes **referential integrity** between tables
`roll_no INT PRIMARY KEY`	`CONSTRAINT fk FOREIGN KEY (dept_id) REFERENCES department(dept_id)`

(H) Syntax for creating a trigger.

```
CREATE [OR REPLACE] TRIGGER trg_name
{BEFORE | AFTER | INSTEAD OF} {INSERT | UPDATE | DELETE} ON table_name
[FOR EACH ROW]
BEGIN
    -- PL/SQL body, using :NEW and :OLD
END;
/
```

Example: `BEFORE UPDATE OF cgpa ON student FOR EACH ROW` when the body logs the old and new value into an audit table.

(I) Define Authorization. The process of granting or revoking **privileges** that decide *what* an authenticated user may access and *do* (SELECT, INSERT, UPDATE, DELETE, EXECUTE, GRANT OPTION), enforced through GRANT/REVOKE and models such as DAC, MAC and RBAC. Authentication = identity; authorization = permission.

(J) What is query optimization? The phase of query processing that chooses the **lowest-cost execution strategy** (access paths, join order, join algorithm, pipelining) among the many logically equivalent plans for a query, using statistics on cardinality, index availability and I/O cost. Done by heuristic (rule-based, e.g. push selections down early) or cost-based (estimated cost, the modern approach) optimizers. Output = execution plan, e.g. visible via `EXPLAIN PLAN FOR`.

---

3. Winter Examination 2024 — full solved paper

Date 20/12/2024 · 3 hours · 100 marks

Q.2 (A) Explain GRANT and REVOKE commands with example. [4]

`GRANT` and `REVOKE` are **DCL (Data Control Language)** commands and implement authorization.

```
GRANT <privileges> ON <object> TO <user|role> [WITH GRANT OPTION];
REVOKE [GRANT OPTION FOR] <privileges> ON <object> FROM <user|role> [CASCADE];
```

- **GRANT** gives a privilege. Object privileges: `SELECT, INSERT, UPDATE(col), DELETE, REFERENCES, ALL`. System privileges: `CREATE TABLE, CREATE SESSION, DBA`. `WITH GRANT OPTION` lets the grantee pass the privilege on.

```
GRANT SELECT, INSERT ON student TO bhavik;
GRANT UPDATE(cgpa) ON student TO faculty WITH GRANT OPTION;
GRANT role_reader TO analytics_user;
```

- **REVOKE** withdraws it. Without `CASCADE`, revoking a privilege that others derived from it is refused; with `CASCADE` the derived privileges are removed too. Revoked privileges take effect immediately and cannot be rolled back.

```
REVOKE INSERT ON student FROM bhavik;
REVOKE UPDATE(cgpa) ON student FROM faculty CASCADE;
```

Hierarchy: `DBA → schema owner → user → role`. Roles are the efficient route — grant privileges to a role, then grant the role to many users, and change once instead of per user.

Q.2 (B) Demonstrate network model. [6]

The network model (CODASYL/DBTG, 1971) generalises the hierarchical model: a record may have **more than one parent**, so the structure is an arbitrary **graph** rather than a tree.

Core concepts: * **Record type** — a collection of fields (like a table/relation). * **Set (owner–member)** — the relationship unit: exactly one owner record type, one or more member record types. A record can participate as a member in many sets → many-to-many support. * **Pointers / linked lists** — each set is physically maintained as a list, so navigation is by pointer chaining (GET NEXT, GET PRIOR) instead of search. This makes traversal fast but the schema brittle. * **DML** is navigational ("find the department record, then get its next employee").

Diagram of the classic structure:

```
          [ DEPARTMENT ]
           /          \
     SET: has        SET: manages
         /               \
  [ EMPLOYEE ] -------- [ PROJECT ]
       \                  /
    SET: assigned --- SET: uses
          \          /
           [ SKILL ]

EMPLOYEE has TWO owners (DEPARTMENT and PROJECT) → allowed here,
impossible in the hierarchical model.
```

Example: `DEPARTMENT 10` owns `EMPLOYEE 101, 102, 103`; `PROJECT P7` also owns `EMPLOYEE 101, 104`. `101` therefore appears in two set instances.

Advantages — represents many-to-many naturally; faster access through pointers; better data integrity/structure rules than a tree; no forced redundancy. **Disadvantages** — complex and hard to design; programs must encode navigation paths, so applications are tied to physical structure; no data independence;

inserting/deleting requires maintaining pointers; no standardisation across vendors (IDS, DMS-1100, Total, IDMS); recursive relationships awkward. Because of these it was replaced by the relational model, but its ideas survive in graph/IMS databases.

Q.2 (C) Illustrate operations of relational algebra. [10]

Relational algebra is a **procedural** language: operators take one or more relations and return a new relation. Unary: σ, π, ρ, δ, τ. Binary set: ∪, ∩, −, ×. Binary special: ⋈ (join), ÷ (division).

Sample data used throughout:

```
STUDENT                          ENROLL
roll | name   | dept | cgpa       roll | course
-----+--------+------+------      -----+-------
101  | Bhavik | CSE  | 8.5        101  | DBMS
102  | Aarav  | CSE  | 7.2        101  | OS
103  | Meera  | ECE  | 9.1        102  | DBMS
```

1. Selection (σ) — horizontal subset; keeps rows satisfying a predicate. `σ_{dept='CSE'}(STUDENT)` → rows 101, 102. Degree unchanged, cardinality ≤ original. Composable: `σ_{cgpa>8 ∧ dept='CSE'}(STUDENT)` → 101.

2. Projection (π) — vertical subset; keeps chosen columns and **automatically removes duplicate rows**. `π_{name, dept}(STUDENT)` → (Bhavik, CSE), (Aarav, CSE), (Meera, ECE). Cardinality may shrink; attribute order can change, so use ρ to fix names.

3. Rename (ρ) — `ρ_{S2}(STUDENT)` renames the relation; `ρ_{(r,n,d,c)}(STUDENT)` renames relation *and* attributes. Needed whenever a binary operator would otherwise create duplicate attribute names.

4. Union (∪) — `R ∪ S` requires **union compatibility**: same degree and same domains in the same order. `π_{roll}(ENROLL WHERE course='DBMS') ∪ π_{roll}(ENROLL WHERE course='OS')` → {101, 102}. Duplicates removed.

5. Set difference (−) — `R − S` = tuples in R not in S. `π_{roll}(STUDENT) − π_{roll}(ENROLL)` → {103} (students with no enrollment). Not commutative.

6. Intersection (∩) — `R ∩ S`; expressible as `R − (R − S)`. → {101}.

7. Cartesian product (×) — `R × S` has degree |R|+|S| and cardinality |R|·|S|. `STUDENT × ENROLL` gives 3×3 = 9 rows. Used as the raw material for joins; on its own it is rarely meaningful.

8. Theta join (⋈θ) — `σ_θ(R × S)`, θ ∈ {=, <, >, ≤, ≠, ≥}. `STUDENT ⋈_{STUDENT.cgpa > 8} ENROLL`.

9. Equi-join — theta join with θ = equality: `STUDENT ⋈_{STUDENT.roll=ENROLL.roll} ENROLL` — keeps both duplicate join columns.

10. Natural join (⋈) — equi-join on all same-named attributes **with the duplicate column removed**. `STUDENT ⋈ ENROLL` →

```
roll | name   | dept | cgpa | course
101  | Bhavik | CSE  | 8.5  | DBMS
101  | Bhavik | CSE  | 8.5  | OS
102  | Aarav  | CSE  | 7.2  | DBMS
```

11. Outer join — preserves unmatched tuples padded with NULL (left/right/full). **12. Division (÷)** — `R ÷ S` = tuples of R (on the extra attributes) that are associated with **every** tuple of S. Use for "and all" queries: students enrolled in *all* courses offered. **13. Aggregate/grouping (γ)** — `γ_{dept; AVG(cgpa)→avg_g}(STUDENT)` → (CSE, 7.85), (ECE, 9.1).

Composite query — "names of CSE students who took DBMS": `π_{name}( σ_{dept='CSE'}(STUDENT) ⋈ σ_{course='DBMS'}(ENROLL) )` → Bhavik, Aarav. Sequencing matters for efficiency: selections are pushed down first — the single most important algebraic optimisation rule.

Q.2 (C — OR branch) Demonstrate DDL and DML commands with detailed syntax and example. [10]

DDL — defines structure.

```
CREATE TABLE department (dept_id INT PRIMARY KEY, dname VARCHAR(20));

CREATE TABLE student (
  roll_no INT PRIMARY KEY, name VARCHAR(30) NOT NULL,
  email   VARCHAR(50) UNIQUE, cgpa DECIMAL(4,2) CHECK (cgpa BETWEEN 0 AND 10),
  dept_id INT, FOREIGN KEY (dept_id) REFERENCES department(dept_id) ON DELETE CASCADE
);

ALTER TABLE student ADD phone VARCHAR(12);
ALTER TABLE student MODIFY name VARCHAR(60);
ALTER TABLE student DROP COLUMN phone;
ALTER TABLE student ADD CONSTRAINT chk_c CHECK (cgpa >= 5);

RENAME student TO student_master;
TRUNCATE TABLE student;      -- deletes all rows, keeps structure, cannot roll back
DROP TABLE student CASCADE CONSTRAINTS;   -- removes table + dependent constraints
DROP TABLE student PURGE;    -- Oracle: skip the recycle bin
```

`DROP` vs `TRUNCATE` vs `DELETE`: DROP removes structure and is DDL; TRUNCATE removes all rows fast, resets storage, DDL, no WHERE, no trigger firing; DELETE is DML, row-by-row, logged, WHERE allowed, fires triggers, and can be rolled back.

DML — manipulates rows.

```
INSERT INTO student VALUES (101,'Bhavik','b@u.ac.in',8.50,3);
INSERT INTO student (roll_no,name,dept_id) VALUES (102,'Aarav',3);
INSERT INTO student SELECT * FROM staged_student WHERE status='OK';

UPDATE student SET cgpa = cgpa + 0.5 WHERE dept_id = 3;
UPDATE student SET dept_id = 4, name = CONCAT(name,' (new)') WHERE roll_no = 102;

DELETE FROM student WHERE cgpa < 4;
DELETE FROM student;                    -- all rows, still transactional

SELECT * FROM student WHERE dept_id = 3 ORDER BY cgpa DESC;

COMMIT; ROLLBACK; ROLLBACK TO sv1;
SAVEPOINT sv1;
SET TRANSACTION READ ONLY;
```

`SELECT` is technically **DQL**; TCL = COMMIT/ROLLBACK/SAVEPOINT; DCL = GRANT/REVOKE. Note that Oracle requires explicit COMMIT while MySQL autocommits — this difference is what Q.3(B) of 2025 probes.

Q.3 (A) Explain GROUP BY and HAVING with syntax and example. [4]

```
SELECT <grouping_cols>, <aggregate_fn>(col)
FROM <table>
WHERE <row_condition>
GROUP BY <grouping_cols>
HAVING <group_condition>
ORDER BY <col> [ASC|DESC];
```

- **GROUP BY** partitions rows into groups sharing the grouping-column values, so aggregates compute **per group** instead of over the whole table. After grouping, the SELECT list may contain only grouping columns

or aggregates — selecting `name` with `COUNT(*)` is an error (indeterminate value).

- **HAVING** filters **groups** (it is applied after aggregation). `WHERE` filters **rows** before grouping and cannot contain an aggregate.

```
SELECT dept_id, COUNT(*) AS strength, AVG(cgpa) AS avg_cgpa
FROM student
WHERE cgpa IS NOT NULL
GROUP BY dept_id
HAVING COUNT(*) > 5 AND AVG(cgpa) > 7
ORDER BY avg_cgpa DESC;
```

Execution order: FROM → WHERE → GROUP BY → aggregates → HAVING → SELECT → ORDER BY → LIMIT. Knowing this order also answers "why can't I use WHERE with COUNT?". `GROUP BY` with `ROLLUP` / `CUBE` / `GROUPING SETS` adds subtotals.

Q.3 (B) Explain basic rules of Armstrong's axioms. [6]

Armstrong's axioms (1974) are a **sound and complete** inference system for functional dependencies — sound because every dependency derived is logically implied; complete because every implied dependency can be derived.

Let U be the attribute set and X, Y, Z ⊆ U.

1. Reflexivity — If Y ⊆ X then `X → Y`. Trivial FDs. Example: `{roll_no, name} → name`; also `X → X` and `X → ∅`. **2. Augmentation (extension)** — If `X → Y` then `XZ → YZ` for any Z. Adding the same attributes to both sides preserves the dependency. Example: from `roll_no → name`, `roll_no, dept → name, dept`. **3. Transitivity** — If `X → Y` and `Y → Z` then `X → Z`. Example: `roll_no → dept_id`, `dept_id → dname` ⟹ `roll_no → dname` (this is exactly the transitive dependency that 3NF removes).

Derived rules (worth writing for full marks): * **Union** — from `X→Y` and `X→Z`, `X → YZ`. * **Decomposition** — from `X → YZ`, `X→Y` and `X→Z`. * **Pseudo-transitivity** — from `X→Y` and `WY→Z`, `XW → Z`. * **Projectivity/Reflexive-transitive**: if `X→Y` and `Z⊆Y` then `X→Z`.

Use — computing the closure X^+. Start with X; repeatedly add right-hand sides whose left side is already in the set. If $X^+ = U$, X is a **superkey**; the smallest such X is the **candidate key**. Example: `F = {A→B, B→C, C→D}`; $A^+ = \{A,B,C,D\}$ ⟹ A is the candidate key. Attribute classification from that: prime attributes appear in some candidate key, non-prime otherwise — and the 2NF/3NF/BCNF tests are built directly on closures.

Q.3 (C) List and explain ACID properties with respect to database transaction. [10]

A **transaction** is a logical unit of work comprising one or more operations on the database that must be treated as an indivisible whole (e.g. fund transfer: `debit A` + `credit B`). ACID is the contract a DBMS must honour, including under crashes and concurrent execution.

A — Atomicity ("all or nothing"). Either every operation of the transaction is reflected in the database, or none is; there is no intermediate visible state. If a failure occurs mid-way, the transaction is **aborted** and all its changes are undone. Guaranteed by the **recovery subsystem** using the log: before-images let the DBMS write an R (rollback) record and restore the old values.

```
BEGIN TRAN;
UPDATE account SET balance = balance - 500 WHERE id = 'A';
UPDATE account SET balance = balance + 500 WHERE id = 'B';
-- power fails here → both updates are undone, money is not lost
COMMIT;
```

C — Consistency. A correct transaction takes the database from one **consistent state** (satisfying all integrity constraints — PK, FK, CHECK, UNIQUE, triggers, business rules such as `balance ≥ 0`) to another consistent

state. Consistency is a property of the *transaction plus constraints*: the DBMS enforces declared constraints and the programmer is responsible for business rules. In the example, consistency means the total money across A and B is unchanged and neither balance goes negative. If the credit fails, an unexecuted pair would leave the sum wrong — so atomicity exists to protect consistency.

I — Isolation (separability). Concurrently executing transactions must not see each other's intermediate results; the effect of any concurrent schedule must be equivalent to some **serial** order of those transactions (**serializability**). Without it the classic anomalies appear: * **Lost update** — two transactions read the same row and both write; the first write is overwritten. * **Dirty read** — reading data another transaction has not committed (and may roll back). * **Non-repeatable read** — re-reading a row after another transaction updated it. * **Phantom read** — re-running a range query after another transaction inserted rows. Enforced by **concurrency control**: 2-Phase Locking (shared/exclusive locks), timestamp ordering, optimistic concurrency control, and MVCC. Isolation levels (`READ UNCOMMITTED → READ COMMITTED → REPEATABLE READ → SERIALIZABLE`) trade isolation strength for concurrency.

D — Durability (recovery). Once a transaction **commits**, its effects survive any subsequent failure — power loss, media failure, crash — permanently. Enforced by the **log-based recovery subsystem**: write-ahead logging (WAL) forces the log record to stable storage before the modified page is written, so the redo pass recovers all committed work; checkpoints bound the work; RAID/mirroring protects against media failure. Implementation: undo/redo log, shadow paging, ARIES.

How the four interlock — table form:

Property	Concern	Failure it defends against	DBMS subsystem
Atomicity	Unit of work	Crash mid-transaction	Log + rollback (recovery)
Consistency	Validity of state	Illegal data / rule violation	Integrity constraints, triggers
Isolation	Interference	Concurrency anomalies	Locking / timestamp / MVCC
Durability	Persistence	Post-commit crash	WAL, checkpoint, redo (recovery)

Additional exam points: a transaction has the states *active → partially committed → committed*, or *failed → rolled back*; `COMMIT` is the point of no return (commit-atomicity problem); the `BEGIN…EXCEPTION…ROLLBACK…END` block in PL/SQL is the programming-level expression of atomicity.

Q.3 (A — OR branch) Explain different types of join. [4]

A **join** combines columns from two or more relations based on a condition.

1. **INNER JOIN** — only rows matching on **both** sides.

```
SELECT s.name, e.course FROM student s INNER JOIN enroll e ON s.roll=e.roll;
```

2. **NATURAL JOIN (⋈)** — auto-matches all identically-named columns and **drops the duplicate**.
3. **EQUI JOIN** — inner join with `=`; keeps both columns (they are not auto-detected).
4. **THETA JOIN (⋈θ)** — non-equality condition: `ON s.cgpa > t.min_cgpa`.
5. **LEFT OUTER** — all left rows; unmatched right columns NULL.

```
101 Bhavik DBMS      102 Aarav DBMS      103 Meera NULL
```

6. **RIGHT OUTER** — mirror image: all right rows kept.
7. **FULL OUTER** — all rows from both sides; NULL-pad wherever there is no match.
8. **CROSS JOIN** — Cartesian product, no condition; |R|×|S| rows.
9. **SELF JOIN** — a table joined to itself via aliases, for hierarchies:

```
SELECT e.name, m.name AS manager FROM emp e LEFT JOIN emp m ON e.mgrid = m.id;
```

10. **NON-EQUI JOIN** — join on a range, e.g. `BETWEEN losal AND hisal` with a salary-grade table.

Rule to remember: outer joins produce NULL-extended tuples, so `COUNT(*)` and `COUNT(col)` differ, and a `WHERE` condition on the optional side silently converts a LEFT JOIN into an INNER JOIN (move it to `ON` to keep the outer behaviour).

Q.3 (B — OR branch) What is data abstraction? Explain with types. [6]

Data abstraction hides the complex internal details of data storage and presentation and exposes only the relevant view; it is what lets non-experts use a database safely and lets developers change internals without breaking applications.

Three levels:

```
External level      : user views / V1, V2, V3   (what each user or app sees)
   | view definitions
Conceptual level    : tables, datatypes, constraints, relationships
   | mapping
Internal/Physical   : records, blocks, indexes, B-trees, hashing, storage layout
```

1. **External / view level** — the highest level; each user sees only the part of the DB relevant to them; hides the rest of the database. Multiple views over the same conceptual schema (student portal vs. fee-office view). Provides security and simplicity.
2. **Conceptual / logical level** — *what* data is stored and the relationships and constraints; defined by the DBA and application designers; independent of storage details. (Historically called the **logical** abstraction.)
3. **Internal / physical level** — *how* data is stored: field representations, record layout, block size, indexes, compression, partitioning.

Corresponding languages: **external/DDL**, **conceptual/DDL**, **internal/physical descriptor**. Mappings between levels are maintained by the DBMS and are what make changes at a lower level invisible above them — this is *logical* and *physical data independence*. Analogy for extra clarity: OOP abstraction — a user calls `balance()` without knowing the table or index it reads.

Q.3 (C — OR branch) Explain ER model in detail. [10]

The **Entity-Relationship model** (Chen, 1976) is a high-level conceptual data model used in the design phase: it describes data in terms the users recognise, before any table exists. Its diagram is the **ERD** / EER diagram.

1. Entity — a real-world distinguishable object about which data is stored (`STUDENT`, `COURSE`, `BOOK`). *Entity set/type* = collection of similar entities = becomes a table. * **Strong entity** — has its own primary key; drawn as a single rectangle. * **Weak entity** — has no key of its own; identified by the owner's key plus a **partial key (discriminator)**, drawn as a double rectangle with a double diamond relationship. Example: `DEPENDENT` identified through `EMPLOYEE`; `TRANSACTION` identified through `ACCOUNT`.

2. Attribute — a property of an entity or relationship; drawn as ellipses. * **Key attribute** — underlined (`roll_no`). * **Simple** — atomic (`age`) vs **Composite** — has components (`name → first, last`, or `address → street, city, pin`). * **Single-valued** (`roll_no`) vs **Multi-valued** — double ellipse (`phone_numbers`, `languages`) → becomes a separate table in mapping. * **Derived** — dashed ellipse (`age` from `dob`, `total_count`); stored only if the DB is denormalised. * **Stored** attributes are the base values.

3. Relationship — association among entities; diamond. **Degree** = number of participating entity sets (unary/binary/ternary). **Role names** label each end (`employee –supervises→ employee`). Cardinality ratio (1:1, 1:N, M:N) and **participation**: **total/mandatory** = double line (every loan must have a borrower), **partial** =

single line. * Identifying relationship (weak entity) uses a double diamond. * Ternary relationships cannot always be reduced to binaries — use them for genuine three-way facts.

4. Specialisation (top-down) — dividing a superclass into subclasses by an inherited attribute or a disjointness/overlap constraint (`STUDENT → UG, PG`). **ISA** triangle; inheritance of attributes and relationships. **Disjoint (`d`)**: an entity is in at most one subclass; **Overlapping (`o`)**: in one or more. Completeness: **total** (double line) vs **partial**. **5. Generalisation (bottom-up)** — merging several entity types that share features into a superclass (`CAR, TRUCK → VEHICLE`); abstract superclass. **6. Aggregation** — treating a *relationship* as an entity so that it can participate in another relationship; used when a binary relationship must itself be related to a third entity: `(EMPLOYEE –WORKS_ON→ PROJECT)` aggregated, then linked to `ENGINEER` with an attribute like `start_date`. **7. Mapping to relational schema (write this to finish the answer)** — strong entity → table; composite → flatten into columns; multi-valued → separate table with FK; 1:N → put the FK on the N side; M:N → new table with both keys as composite PK; weak entity → table with PK = owner PK + discriminator; ternary → table with the three FKs.

A worked mini-model: `STUDENT(roll_no PK, name, cgpa)` —1:N— `DEPARTMENT(dept_id PK, dname)`; `STUDENT` M:N `COURSE` via `ENROLL(roll, course, semester, grade)`; `TRANSACTION` as a weak entity of `ACCOUNT`.

Q.4 (A) Explain any 4 numeric functions with syntax and example. [4]

Function	Syntax	Result
ABS	`ABS(n)`	`ABS(-25) = 25`
CEIL	`CEIL(n)` smallest integer ≥ n	`CEIL(4.1) = 5`, `CEIL(-4.8) = -4`
FLOOR	`FLOOR(n)` largest integer ≤ n	`FLOOR(4.9) = 4`
ROUND	`ROUND(n [, d])` rounds to d decimals	`ROUND(456.789, 2) = 456.79`, `ROUND(456.789) = 457`, `ROUND(1234,-2)=1200`
TRUNC	`TRUNC(n [, d])` cuts off, no rounding	`TRUNC(456.789, 2) = 456.78`
MOD	`MOD(m, n)` remainder	`MOD(10, 3) = 1`
POWER	`POWER(x, y)`	`POWER(2,10) = 1024`
SQRT	`SQRT(x)`	`SQRT(81) = 9`
SIGN	`SIGN(n)` → 1/0/−1	`SIGN(-15) = -1`

```
SELECT name, ROUND(cgpa,1) AS score, CEIL(cgpa) AS band, MOD(roll_no,2) AS parity
FROM student;
```

Q.4 (B) Compare sparse and dense index. [6]

An **index** is an auxiliary access structure: a file (usually a B/B$^+$-tree) of index records holding `<search-key value, pointer to data record>`. Let the data file have **n** blocks and **r** records.

Aspect		
Entries	One index record for **every** search-key value in the file	Index records for **only some** key values — typically the first key of each data block
Requirement	None; works for any file order	Data file must be **sorted and ordered** on the search key (primary/clustered file)
Size	Large: n index records → more blocks and more I/O	Small: far fewer records → fewer levels, less I/O
Search	Direct: locate the exact key, follow pointer	Binary search within the index to find the largest key ≤ target, then **scan the data block** to confirm
Range query	Excellent — keys are stored in order and contiguous	Good on the ordering key; must scan blocks
Insertion cost	High — must add an entry for every new row	Lower — only new block starts need entries
Secondary index	**Must** be dense (a non-ordering key can't be sparse)	Only possible on the ordering/primary key

Both are further layered into **multi-level indexes** so an index fits in a few blocks: an outer index over the inner index's block pointers. `n=20,000`, `b=100` records/block → dense = 20,000 index records ≈ 200 blocks, sparse = 200 records ≈ 2 blocks. That numeric contrast is worth including. Exam cross-check: a **primary index** is normally sparse (ordering key of a densely stored file), a **secondary/alternate index** is always dense, and a **clustered index** exists only once per table.

Q.4 (C) Explain normalization with its types. [10]

Normalization is a bottom-up design process that decomposes a relation into smaller relations to eliminate **data redundancy** and the three **anomalies**: *insertion* (cannot record a fact without unrelated facts), *update* (one change must be repeated in many rows), *deletion* (removing a row loses other facts). It is formalised with **functional dependencies** and normal forms.

Bad starting relation (one row per student per subject):

```
STUDENT_SUB(roll_no, name, dept_id, dname, course_id, cname, marks)
101 Bhavik 3 CSE DBMS Database 85
101 Bhavik 3 CSE DBMS Database 85   → name/dept/dname repeated for every subject
```

Anomalies: a new department cannot be added without a student (insertion); changing the HOD requires updating every row (update); dropping the last CSE row loses the department (deletion).

1NF — atomic values only; no repeating/multi-valued groups; a PK identifies each row. `phone` holding `"98,76"` violates it → move to a separate table. *Example*: `STUDENT_SUB` is already in 1NF.

2NF — in 1NF **and** no **partial dependency**: every non-prime attribute must depend on the *whole* candidate key, not part of it. Applies when the PK is composite. Here `roll_no → name` depends on part of the key `(roll_no, course_id)` → violates 2NF. Decompose: `R1(roll_no, name, dept_id)` + `R2(roll_no, course_id, marks)`, and `dname` depends on `dept_id` → `R3(dept_id, dname)`.

3NF — in 2NF **and** no **transitive dependency** among non-prime attributes (Codd). Equivalently the standard test used in exams: for every FD `X → A`, either (i) X is a superkey, or (ii) A is a prime attribute. `dept_id → dname` with `roll_no → dept_id` is transitive → violates 3NF in `R1`. Split: `R1(roll_no, name, dept_id)` + `R3(dept_id, dname)`.

BCNF (E.F. Codd's enhanced form) — for **every** non-trivial FD `X → A`, X must be a **superkey**. Stricter than 3NF: it allows no determinant that is not a candidate key, and it can fail to be dependency-preserving. Case

where 3NF holds but BCNF fails: `TEACH(student, teacher, subject)` with `student,subject → teacher`, `teacher → subject`. Candidate keys `{student,subject}` and `{student,teacher}`; `teacher → subject` has a non-superkey determinant on a *prime* dependent → 3NF-legal, BCNF-illegal. Decompose into `R1(teacher, subject)` and `R2(student, teacher)`.

4NF — in BCNF and **no non-trivial multi-valued dependency** `X ↠ Y` where X is not a superkey. `PROFESSOR(subject, book, course)` where subject determines a set of books and a set of courses independently → split into `P-S(subject, book)` and `P-C(subject, course)`.

5NF (PJNF/Project-Join NF) — in 4NF and every **join dependency** of R is implied by a candidate key: R cannot be losslessly decomposed into smaller relations that are only jointly meaningful. Rare in practice; example: supplier–part–project triples with independent constraints. Also mention **DOMAIN/KEY NF (6NF)** — no attribute may overlap a domain with a key (temporal databases), theoretical only.

Canonical result:

```
STUDENT(roll_no PK, name, dept_id FK)
DEPARTMENT(dept_id PK, dname)
COURSE(course_id PK, cname)
ENROLL(roll_no, course_id PK, marks)     -- resolves the M:N
```

Process: list FDs → find candidate keys via closure → fix 1NF (atomics) → remove partial deps → remove transitive deps → check BCNF → check MVDs/JDs → verify **lossless-join** (common attribute is a key of one side) and **dependency-preserving** (all FDs recoverable on the decomposed relations). **Denormalization** is the deliberate reverse step taken in data warehouses for read performance.

Q.4 (A — OR branch) Explain any 4 character functions with syntax and example. [4]

Assume `name = 'database management'`.

Function	Syntax	Result
UPPER	`UPPER(s)`	`'DATABASE MANAGEMENT'`
LOWER	`LOWER(s)`	`'database management'`
INITCAP	`INITCAP(s)`	`'Database Management'`
CONCAT	`CONCAT(s1, s2)` or `||`	`CONCAT('data','base') = 'database'`
SUBSTR	`SUBSTR(s, pos [, len])`, pos starts at 1, negative counts from end	`SUBSTR('database',4,3) = 'aba'`
LENGTH	`LENGTH(s)` characters (`LEN` in SQL Server)	`LENGTH('database') = 8`
LPAD / RPAD	`LPAD(s, n [, pad])`	`LPAD('123',6,'0') = '000123'`
LTRIM/RTRIM/TRIM	`TRIM([lead FROM] s)`	`TRIM(' db ') = 'db'`
REPLACE	`REPLACE(s, from, to)`	`REPLACE('cat','at','ow') = 'cow'`
INSTR	`INSTR(s, sub)` position of first occurrence	`INSTR('database','a') = 2`
REVERSE	`REVERSE(s)`	`'egnemtnagem etabased'`

```
SELECT UPPER(name), LENGTH(name), SUBSTR(name,1,3) AS code, LPAD(roll_no,5,'0') AS padded
FROM student WHERE name LIKE 'B%';
```

Note `LENGTH('')` is NULL in Oracle but 0 in MySQL, and Oracle `SUBSTR` is 1-based while some APIs are 0-based — a common MCQ trap.

Q.4 (B — OR branch) What is view? Explain its types. [6]

A **view** is a virtual table: it stores no data of its own, only a saved query; the base tables supply rows at query time. It is used for security (column/row restriction), simplifying complex joins, presenting a stable interface while the schema changes, and aggregating data.

```
CREATE VIEW v_cse_toppers AS
SELECT roll_no AS student_id, name, cgpa
FROM student WHERE dept_id = 3 AND cgpa >= 8
WITH CHECK OPTION;

SELECT * FROM v_cse_toppers WHERE cgpa > 9;
DROP VIEW v_cse_toppers;
```

Types 1. **Simple / single-table view** — one table, no joins or groups; **updatable** (INSERT/UPDATE/DELETE pass through), provided it keeps the PK and no aggregate/DISTINCT/GROUP BY appears. 2. **Complex / multi-table view** — joins, groups, subqueries, functions; usually **read-only**. 3. **Joined view** — built over two or more joined tables (student + department). 4. **Aggregated / grouped view** — `SELECT dept_id, AVG(cgpa) … GROUP BY` — not updatable. 5. **Projection view** — subset of columns; used for column-level security. 6. **Materialized view** — the result **is physically stored** and refreshed (COMPLETE/FAST/on-commit/on-demand refresh); used in data warehousing to trade space for speed. Ordinary views are recomputed on every use. 7. **View with CHECK OPTION** — prevents DML through the view that would create rows the view cannot show, preserving the defining predicate.

Limitations: not all views are updatable; the definition breaks if the base table changes; nested views add optimizer complexity; a plain view gives no performance gain.

Q.4 (C — OR branch) What is trigger? Explain its types. [10]

A **trigger** is a stored PL/SQL (or SQL) block that the DBMS executes **automatically** in response to a specified DML/DDL event on a table or view; the user does not call it. It is the mechanism for enforcing business rules and referential/integrity constraints that ordinary declarations cannot express, and for auditing.

```
CREATE [OR REPLACE] TRIGGER trigger_name
{BEFORE | AFTER | INSTEAD OF} {INSERT OR UPDATE [OF col] OR DELETE} ON table_name
[REFERENCING NEW AS n OLD AS o]
[FOR EACH ROW] [WHEN (condition)]
BEGIN  ... body using :NEW / :OLD ...  END;
/
```

Row-level triggers see `:NEW.col` / `:OLD.col` (and may modify `:NEW.col` in a BEFORE trigger); statement-level triggers see neither.

Classification

Basis	Types
Timing	`BEFORE` — validate/derive values before the change is applied (cheapest way to reject bad data) · `AFTER` — act on the completed change (audit, cascading updates) · `INSTEAD OF` — defined **on a view**, replaces the DML so a non-updatable view can be written to
Event	`INSERT` (`:OLD` unusable) · `UPDATE` (both, plus `OF col` to fire only for chosen columns) · `DELETE` (`:NEW` unusable)
Granularity	`STATEMENT` level — fires **once per SQL statement** (default) · `ROW` level — `FOR EACH ROW`, once per affected tuple, e.g. 10,000 updates → 10,000 firings
Order / condition	`FOLLOWS/PRECEDES` for ordering same-event triggers; `WHEN (condition)` filters firing without writing an IF

Worked example — enforce the business rule cgpa ∈ [0,10] and log changes:

```
CREATE TRIGGER trg_cgpa_check BEFORE UPDATE OF cgpa ON student FOR EACH ROW
BEGIN
  IF :NEW.cgpa NOT BETWEEN 0 AND 10 THEN
     RAISE_APPLICATION_ERROR(-20001,'cgpa out of range');
  END IF;
END; /

CREATE TRIGGER trg_audit AFTER UPDATE ON student FOR EACH ROW
BEGIN
  INSERT INTO audit_log(roll_no, old_val, new_val, changed_by, when_at)
  VALUES (:OLD.roll_no, :OLD.cgpa, :NEW.cgpa, USER, SYSDATE);
END; /

CREATE TRIGGER trg_v_insert INSTEAD OF INSERT ON v_cse_toppers FOR EACH ROW
BEGIN INSERT INTO student(roll_no,name,cgpa) VALUES (:NEW.student_id,:NEW.name,:NEW.cgpa); END; /
```

Types by purpose (write this list too): `BEFORE row`, `BEFORE statement`, `AFTER row`, `AFTER statement`, `INSTEAD OF` (view), **compound trigger** (`BEFORE STATEMENT … BEFORE EACH ROW … AFTER EACH ROW … AFTER STATEMENT` in one unit, which avoids the mutating-table error), **DDL triggers** (`BEFORE DDL / AFTER DDL ON DATABASE` or `ON SCHEMA`, e.g. blocking `DROP TABLE`) and **logon/logoff triggers**. Cautions: triggers are invisible so they surprise maintainers; a trigger that fires another trigger can loop (guarded by `RAISE_APPLICATION_ERROR` or a firing-depth check); `ORA-04091 mutating table` arises when a row trigger queries its own table; excessive triggers slow bulk DML. MySQL supports triggers only since 5.0 and only `FOR EACH ROW`.

Q.5 (A) Explain time-stamp based protocol. [4]

Timestamp ordering is a **non-locking** concurrency-control scheme that guarantees conflict-serializability by executing transactions in the order of their timestamps. Each transaction gets a unique, monotonically increasing timestamp TS(T) at start (a counter or the system clock). Each data item Q carries two stamps: **W-timestamp(Q)** = TS of the last transaction that wrote Q, **R-timestamp(Q)** = TS of the last transaction that read Q.

Rules — a read/write is allowed only if it would not violate the timestamp order; otherwise the transaction is **rolled back**: * **Read rule** — if `TS(Ti) < W-ts(Q)`, T wants to read a value already overwritten by a later transaction → reject and roll back Ti. Else allow, set `R-ts(Q) = max(R-ts(Q), TS(Ti))`. * **Write rule** — if `TS(Ti) < R-ts(Q)` (a later transaction already read the old value) → reject. If `TS(Ti) < W-ts(Q)` (a later transaction already wrote it) → reject (obsolete write). Else allow, set `W-ts(Q) = max(W-ts(Q), TS(Ti))`.

Example: `W-ts(Q)=12`, `R-ts(Q)=15`. `T10` reads Q → allowed (10 ≥ … value still valid? no: 10 < W-ts 12 → rejected, T10 rolls back). `T17` writes Q → allowed since 17 > both stamps.

Refinements — *Thomas' write rule* removes the obsolete-write rejection by discarding the stale write instead of aborting, so more schedules are recoverable. *Strict 2PL comparison*: timestamp schemes avoid **deadlock entirely** (nothing is held), but can cause repeated aborts and starvation, and they may allow cascading aborts unless combined with strict locking; they also need the system to keep all timestamps and to restart aborted transactions with a **new** timestamp. Chained/conservative timestamp ordering and multiversion timestamping (MVTO) reduce aborts.

Q.5 (B) Construct PL/SQL block to take 3 values from user and find maximum from 3 values. [6]

```
SET SERVEROUTPUT ON;

DECLARE
    a   NUMBER := &first;          -- substituted by SQL*Plus at run time
    b   NUMBER := &second;
    c   NUMBER := &third;
    mx  NUMBER;
BEGIN
    mx := a;
    IF b > mx THEN mx := b; END IF;
    IF c > mx THEN mx := c; END IF;

    DBMS_OUTPUT.PUT_LINE('Maximum of '||a||', '||b||', '||c||' = '||mx);
EXCEPTION
    WHEN VALUE_ERROR THEN DBMS_OUTPUT.PUT_LINE('Invalid numeric input.');
END;
/
```

Execution with input `12 45 33` prints `Maximum of 12, 45, 33 = 45`.

Equivalent using `GREATEST` (one built-in call): `SELECT GREATEST(&a,&b,&c) FROM dual;`. Alternative structure with `ELSIF` and `DECODE`/nested `IF…ELSE` is equally acceptable; mark allocation goes to declaration, correct substitution syntax, the comparison logic, output, and the terminating `/`.

Q.5 (C) Illustrate hashing with its types. [10]

Hashing gives direct access to a record from its key without an index tree. A **hash function** h maps the search key to a **bucket address** in the file; the record is stored there. Cost ≈ 1–2 block accesses regardless of file size — optimal for equality lookups (`WHERE roll_no = 101`), but poor for range queries (`BETWEEN`), which is exactly what ordered B/B$^+$-tree indexes serve. Components: the hash function (division `k mod p`, multiplication, mid-square, digit analysis), the **bucket** (one or more blocks), and a **overflow/chaining** policy for collisions.

```
h(k) = k mod 7
key 101 → 101 mod 7 = 3 → bucket 3
```

Types

1. Internal (in-memory) vs External (disk) hashing — internal hashing = hash table for a small relation kept in a memory table; external/disk hashing = buckets are disk blocks, used for very large relations.

2. Open (non-bucket) addressing — no overflow chains; on collision, probe other slots in the file itself by a fixed sequence: *linear probing* (h+1, h+2, …), *quadratic probing* (h+1^2, h+2^2, …), *double hashing* (h + i·h$_2$(k)). Works only when the table is sparsely filled (load factor $\lesssim 0.7$); clustering degrades it. Used for in-memory structures.

3. Closed addressing / separate (overflow) chaining — each bucket holds a pointer to a chain of overflow blocks containing the collided records. * **Fixed-address chaining** — a pre-allocated contiguous overflow area. * **Overflow-chaining** — one overflow block per bucket, chained by pointers. * **Reorganization** — periodically

redistribute the bucket + its chain to keep chains short. Search cost = 1 block + average chain length; it is the most common disk technique.

4. Static (deterministic/closed) hashing — the number of buckets N is fixed by the file definition, so `h(k) = k mod N` never changes. Simple and predictable, but when the file grows, chains lengthen and performance falls; enlarging N requires **rehashing the whole file**.

5. Dynamic (extendible) hashing — the file grows **gradually** without rehashing everything. * Uses a **global depth** (number of bits hauled from the key) and a **directory** of `2^gd` entries, each pointing to a bucket with its own **local depth**. * On overflow of a bucket with local depth < global depth, split it into two, redistribute its records by the next bit, and update the directory entries. * If local depth = global depth, the **directory doubles** (rare, cheap: just pointers). * Handles insertion *and* deletion, and shrinks when buckets empty. * **Linear hashing** is the variant that splits buckets in a fixed round-robin order without a directory, doubling the file lazily — good when the final size is unknown.

Comparison table to finish:

	Static	Extendible (dynamic)	Linear
Buckets change?	No	Yes, via directory	Yes, lazily, no directory
Rehash all data?	On resize: yes	No	No
Directory	None	`2^gd` pointers	None
Range queries	No	No	No

Hashing vs B^+-tree: equality access is O(1) vs O(log n), but no ordering, no range scan, and it needs rebalancing/split policies. In practice a **hash index** is used on the primary key of huge append-heavy tables and a B^+-tree where range/order matters.

Q.5 (A — OR branch) Demonstrate structure of PL/SQL block. [4]

PL/SQL = **Procedural Language extension of SQL** (Oracle). It blends SQL data access with procedural control flow, executes as one unit (fewer context switches), and is block-based, portable, and exception-aware.

```
DECLARE            -- OPTIONAL: variables, constants, cursors, types
   v_name  student.name%TYPE;
   c_age   CONSTANT NUMBER := 18;
   CURSOR cur IS SELECT name FROM student;
BEGIN              -- MANDATORY: executable section, SQL + PL/SQL statements
   SELECT name INTO v_name FROM student WHERE roll_no = &r;
   IF v_name IS NOT NULL THEN DBMS_OUTPUT.PUT_LINE('Hello '||v_name); END IF;
EXCEPTION          -- OPTIONAL: runtime error handlers
   WHEN NO_DATA_FOUND THEN DBMS_OUTPUT.PUT_LINE('No such student');
   WHEN TOO_MANY_ROWS THEN DBMS_OUTPUT.PUT_LINE('More than one row');
   WHEN OTHERS        THEN DBMS_OUTPUT.PUT_LINE('Error: '||SQLERRM);
END;               -- terminator
/
```

Rules: only BEGIN…END is mandatory; `:=` assigns in PL/SQL while SQL uses `=`; `%TYPE` / `%ROWTYPE` give schema-anchored declarations; `&var` prompts at run time; statements end with `;` and the block ends with `/`. Named blocks (`<<label>> … END label;`) allow controlled exit from nested loops.

Components worth naming: sub-blocks, scope rules (inner declarations shadow outer), and the schema objects PL/SQL can hold — procedures, functions, packages, triggers, cursors, and collections. **Advantages**: modular, faster (one round trip), secure (grant execute on the program, not on the tables), error-tolerant via EXCEPTION, portable across Oracle versions. **Disadvantages**: proprietary to Oracle, harder to debug, ties logic to the database tier.

Q.5 (B — OR branch) Construct PL/SQL block to take 3 values from user and find its sum and average. [6]

```
DECLARE
    a     NUMBER := &first;
    b     NUMBER := &second;
    c     NUMBER := &third;
    s     NUMBER;
    avg_v NUMBER(10,2);
BEGIN
    s     := a + b + c;
    avg_v := ROUND(s / 3, 2);
    DBMS_OUTPUT.PUT_LINE('Sum    = ' || s);
    DBMS_OUTPUT.PUT_LINE('Average = ' || avg_v);
END;
/
```

Input `10 20 30` → `Sum = 60`, `Average = 25`. One-line alternative that markers also accept: `SELECT &a + &b + &c AS sum, (&a + &b + &c)/3 AS average FROM dual;`. Write the block version as the main answer — the question asks for a *PL/SQL block*, so DECLARE/BEGIN/END and `/` carry the marks. `avg` is close to a reserved word, hence the variable name `avg_v`.

Q.5 (C — OR branch) Construct a B-tree for (16,3,15,7,1,5,18,9,35,28,39,17,14,20,22) with order 5. [10]

Convention used: order 5 = at most **5 children** per node = at most **4 keys**; a node splits when a 5th key arrives; root may have ≥1 key, other nodes ≥ ⌈5/2⌉−1 = 2 keys.

Insertion sequence and the two splits:

Insert order	Keys	What happens
1–4	16, 3, 15, 7	single leaf, filled in sorted order → `[3, 7, 15, 16]` (4 keys = full, still legal)
5	1	leaf overflows with `[1, 3, 7, 15, 16]` → median **7** promoted, tree becomes root `[7]` with leaves `[1,3]` and `[15,16]`
6–8	5, 18, 9	both leaves absorb them, no split → `[1,3,5]` and `[9,15,16,18]`
9	35	right leaf `[9,15,16,18,35]` overflows → median **16** promoted → root `[7,16]`, leaves `[9,15]` and `[18,35]`
10–11	28, 39	third leaf fills → `[18,28,35,39]`
12	17	that leaf becomes `[17,18,28,35,39]` and overflows → median **28** promoted → root `[7,16,28]`, leaves `[17,18]` and `[35,39]`
13–15	14, 20, 22	remaining inserts just fill leaves; `[17,18,20,22]` is exactly full — final tree below

Final B-tree (verified by construction):

```
level 0 (root)         [ 7 | 16 | 28 ]
                       /    |    |    \
level 1 (leaves)  [1,3,5] [9,14,15] [17,18,20,22] [35,39]
```

Properties check: every leaf holds ≤ 4 keys and ≥ 2 (except that no leaf is under-full — 2,3,4,2 ✓); root has 3 children ✓; keys increase left to right and each subtree contains exactly the range delimited by its parent's

separators (< 7, 7–16, 16–28, ≥ 28) ✓; all leaves at the same depth (height 2) ✓; 15 keys total ✓.

How to present the answer in the exam: draw the initial empty node, then show the tree **after every split** with an arrow marking the promoted key. The marks are for the split steps and the median choice, not only the final picture. Also state the two rules you applied (max keys = order−1, split at order keys, median goes up).

Note on the order convention: some textbooks read "order m" as **m keys** per node rather than m children. Under that reading the same sequence yields a tree of height 2 as well but with 6 keys per node. If your notes use the Silberschatz definition (max m children) the tree above is the expected answer; state the convention explicitly at the top of your solution so the marker knows.

---

4. Winter Examination 2025 — full solved paper

Date 08/12/2025 · 3 hours · 100 marks

Because Q.2(C), Q.3(B), Q.3(C), the join branch, the network-model branch and the "max of three" branch are the **same questions as 2024**, they are cross-referenced with their answer skeleton instead of being reprinted in full. Write the full version in the exam.

Q.2 (A) Discuss data independence with its types. [4]

Data independence is the capacity to modify the schema at one abstraction level of a DBMS without having to change the schema or the application programs at the next higher level. It exists because the three levels (internal, conceptual, external) are separated by **mappings maintained by the DBMS**, which shield each level from the one below. It is the central advantage of a DBMS over a file-processing system, where a change in record layout forces every program that reads the file to be rewritten.

1. Logical data independence — changing the **conceptual schema** without changing the external level or application programs. Operations: adding a new table or attribute, extending the length of a field, adding a new entity/relationship type, splitting a table into two, renaming a column. Because views are defined by queries over the conceptual schema, they keep working as long as the attributes they reference are not removed. *Example*: adding `middle_name` to `student`, or creating a new `scholarship` table — the fee-module query `SELECT roll_no, cgpa FROM student` is untouched. *Harder to achieve* than physical independence, because the semantic definition of the data itself is what changes; deleting or renaming a referenced column usually breaks views, which is why views must be redefined.

2. Physical data independence — changing the **internal/storage schema** without affecting the conceptual or external level. Operations: changing file organisation (heap → hash → indexed), adding or dropping an index, moving to a different storage device, changing block size, compression or encryption, partitioning a table, upgrading DBMS hardware/version. *Example*: creating `CREATE INDEX idx_dept ON student(dept_id)` — the query text and results are identical, only faster; the optimizer picks the index internally.

	Logical DI	Physical DI
Level changed	Conceptual	Internal/physical
Typical change	new table/column, split relation	index, file org, storage, partitioning
Programs affected	Should be none (views may need redefinition)	None
Ease	Difficult	Relatively easy

A third term occasionally quoted is **external/view-level independence**, satisfied by redefining a view without changing base tables — the reason applications are written against views.

Q.2 (B) Define features, applications and users of DBMS. [6]

A **Database Management System** is system software that defines, creates, maintains and controls access to a database, providing an abstraction layer between the data and the applications.

Features (characteristics) 1. **Self-describing catalogue** — stores both data and **metadata** in the system catalogue/dictionary; the DBMS can inspect its own definition, which a file system cannot. 2. **Data abstraction & 3-level architecture** — internal/conceptual/external levels with data independence. 3. **Data sharing & multi-user support** — concurrent access with concurrency control so users do not interfere (lock manager). 4. **Minimal/reduced data redundancy** — integration, not duplication; the same fact stored once (but redundancy is allowed deliberately for performance). 5. **Data integrity** — entity, referential and user-defined integrity (PK, FK, NOT NULL, UNIQUE, CHECK, triggers). 6. **Data security** — authentication, authorization via GRANT/REVOKE, views, encryption, auditing. 7. **Transaction management & ACID** — atomic units of work, crash recovery from the log. 8. **Backup and recovery** procedures, checkpointing, point-in-time restore. 9. **Data independence** — logical and physical. 10. **Language interfaces** — DDL, DML/DQL, DCL, TCL, embedded SQL, PL/SQL, APIs (ODBC/JDBC/REST). 11. **Data constraints & enforcement** automatically rather than in each program. 12. **Multiple views of the same data** for different users. Compared with file systems, it also gives centralised control, eliminates program-data dependence, and allows standardisation — its costs are price, complexity, and a single point of failure needing protection.

Users (actors) * **DBA (Database Administrator)** — the power centre: defines the schema, grants privileges, monitors performance, tunes indexes, manages storage, plans backup/recovery, controls concurrency and availability. * **Database designers** — decide the logical/physical structure: tables, attributes, constraints, normalisation, indexes (often with the DBA). * **Application programmers / developers** — write the software that accesses the DB via SQL, embedded SQL, JDBC/ODBC, ORM, stored procedures and triggers; they never touch physical storage. * **Naive / unsophisticated end users** — interact through fixed GUI forms and menus (ATM, web portal, mobile app); they don't know SQL. * **Interactive / sophisticated end users** — analysts, scientists, data analysts who write ad-hoc SQL queries directly. * **Parametric end users** — use pre-defined parameterised transactions (bank teller entering amount and account). * **Specialised users** — CAE engineers, AI/expert-system and graphics users, mobile/web users. * Occasionally listed: **system analysts** who study requirements and specify what data and processing is needed.

Applications (where DBMS is used) Banking (accounts, loans, transactions — the classic ACID case), finance and stock exchanges, airlines and railway **reservation** (high concurrency, integrity), e-commerce and retail/inventory (POS + stock), telecommunications and billing, universities (students, courses, enrollment, results), hospitals and healthcare records, HR/payroll, search engines and social networks, manufacturing and supply-chain/ERP, data warehousing and business intelligence, government (voter ID, census), multimedia/GIS, and every web application's back end. Each is chosen to illustrate a different requirement: concurrency, integrity, volume, or complex querying.

Q.2 (C) Explain all operations of relational algebra. [10]

Identical to 2024 Q.2(C) — see §3 for the complete answer with worked sample data. Skeleton for a 10-mark answer, one line + one example per operator: σ selection (rows) → π projection (columns, de-duplicates) → ρ rename → ∪ union, − difference, ∩ intersection (all require union compatibility) → × Cartesian product (degree adds, cardinality multiplies) → ⋈θ theta join, equi-join, natural join (duplicates removed), outer joins → ÷ division ("all"/"every" queries) → γ grouping/aggregation → and one composite query. Close with: "these operators are closed — the result of every operation is itself a relation — which is why they can be nested; and the optimizer's most important rewrite is pushing σ and π as low as possible."

Q.2 (C — OR branch) Demonstrate DDL and DML commands. [10]

Same as 2024 Q.2(C) OR branch — see §3. Write: DDL block (CREATE with all constraint types, ALTER ADD/MODIFY/DROP, TRUNCATE vs DROP vs DELETE table, RENAME), DML block (INSERT single/multi/`INSERT…SELECT`, UPDATE with subquery, DELETE, SELECT with WHERE/ORDER BY), TCL (COMMIT/ROLLBACK/SAVEPOINT), DCL (GRANT/REVOKE), and a comparison table of the four language classes.

Q.3 (A) Explain network model. [4]

Same as 2024 Q.2(B) — see §3 for the diagram and full discussion. Four-mark version: CODASYL/DBTG graph model; record types and **sets** with one owner and many members; a record may have **multiple parents**, giving native many-to-many; access is **navigational through pointers/linked lists** (GET NEXT, GET PRIOR); advantages = represents complex relationships and fast traversal; disadvantages = complex design, program-data dependence (paths hard-coded), no data independence, pointer maintenance, poor standardisation; superseded by the relational model.

Q.3 (B) Explain basic rules of Armstrong's axioms. [6]

Verbatim identical to 2024 Q.3(B) — reproduce that answer. Reflexivity, augmentation, transitivity (the three basic rules, each with an example), plus union, decomposition and pseudo-transitivity as derived rules; soundness and completeness; and the closure algorithm X^+ used to find candidate keys.

Q.3 (C) List and explain ACID property with respect to transaction. [10]

Verbatim identical to 2024 Q.3(C). Atomicity → log/rollback; Consistency → integrity constraints and states; Isolation → serializability and the four anomalies, enforced by 2PL/timestamp/MVCC; Durability → WAL, checkpoint, redo. Include the fund-transfer code fragment and the summary table of property ↔ subsystem.

Q.3 (B — OR branch) Discuss MySQL and Oracle database. [6]

Aspect		
Nature/License	Open source (GPL), owned by Oracle Corp.; forks MariaDB, Percona	Commercial, closed source; expensive licensing; free XE edition
Cost & target	Free; web apps, startups, LAMP stack	Enterprise; banking, ERP, large OLTP/DW
Storage engines	**Pluggable**: InnoDB (row-level locks + MVCC), MyISAM (table locks), Memory, Aria — chosen per table	Single unified engine (plus options in 23c)
Procedural language	Stored procedures/functions/triggers supported, weaker type system	**PL/SQL** — richer: packages, object types, collections, records, bulk operations, %TYPE/%ROWTYPE
Data dictionary	`INFORMATION_SCHEMA`	`USER_/ALL_/DBA_TABLES`, `V$` dynamic views
LIMIT syntax	`LIMIT n OFFSET m`	`ROWNUM`, and 12c+ `FETCH FIRST n ROWS ONLY` / `OFFSET`
Auto-increment	`AUTO_INCREMENT` column	`SEQUENCE` + trigger, or 12c `IDENTITY` column
Booleans	`BOOLEAN` / `TINYINT(1)` native	Historically no BOOLEAN type (use `NUMBER(1)`; BOOLEAN added in 23c)
COMMIT behaviour	**Autocommit ON by default** in most clients	Autocommit **OFF** by default; explicit `COMMIT` required
Schemas	In Oracle terminology a MySQL "database" *is* the schema; no separate schema-user split	Schema is owned by a user; one instance hosts many schemas
Concurrency control	InnoDB row locks + gap/next-key locks, MVCC via undo log	Row-level locking, **read-consistency via undo/rollback segments**, latch & enqueue machinery, block/change tracking
Recovery/HA	Replication (binlog), Group Replication, InnoDB crash recovery	RMAN, Data Guard, Flashback, Real Application Clusters (RAC) — true shared-disk clustering
Partitioning/clustering	Basic partitioning free; sharding via middleware	Advanced partitioning, RAC parallel execution across nodes
Optimizer	Cost-based (since 5.x), histogram support	Mature CBO with statistics, SQL Plan Management, hints
Portability/standards	Simpler, more forgiving SQL; silently converts bad data in non-strict mode	Stricter typing; `sql_mode=STRICT` equivalent is the default
Load data	Fast bulk `LOAD DATA INFILE`	SQL*Loader, External tables, Data Pump

Conclusion to write: MySQL wins on cost, simplicity and web-scale read workloads; Oracle wins on transactional integrity, procedural power, scalability (RAC), security/auditing and enterprise support. Both are relational, SQL-compliant and ACID when configured (InnoDB for MySQL).

Q.3 (C — OR branch) Describe components of ER diagram. [10]

Nearly identical to 2024 Q.3(C) ER-model answer. For the "components" wording, lead with the shape legend — that is what earns the marks:

Component	Symbol	Variants
Entity (strong)	Rectangle	—
Weak entity	**Double** rectangle	needs owner + partial key
Relationship	Diamond	identifying = **double** diamond
Attribute	Ellipse	key = underline; multi-valued = double ellipse; derived = dashed ellipse; composite = nested ellipses
Partial key / discriminator	Underline with dotted ellipse	e.g. `Dep_name` of DEPENDENT
Line / link	Single line	**double line** = total participation
Cardinality ratio	(1:1, 1:N, M:N) on the lines	with min-max notation (0,N) (1,1)
ISA (specialisation)	Triangle	`d` disjoint, `o` overlapping
Aggregation	Dashed box around a relationship	lets a relationship be an entity
Generalisation	Reverse ISA	bottom-up

Then define each, give the weak-entity and multi-valued examples, distinguish participation vs cardinality, explain specialisation/generalisation/aggregation with an example each, and finish with the mapping rules to tables.

Q.4 (A) Construct a PL/SQL block that will take 3 values from user and find which is maximum from them. [4]

Same question as 2024 Q.5(B) — reproduce it with the substitution variables and `DBMS_OUTPUT`:

```
DECLARE
   a NUMBER := &num1;  b NUMBER := &num2;  c NUMBER := &num3;  mx NUMBER;
BEGIN
   mx := a;
   IF b > mx THEN mx := b; END IF;
   IF c > mx THEN mx := c; END IF;
   DBMS_OUTPUT.PUT_LINE('Maximum = ' || mx);
EXCEPTION WHEN OTHERS THEN DBMS_OUTPUT.PUT_LINE('Error: ' || SQLERRM);
END;
/
```

Q.4 (B) Explain transactional control commands. [6]

TCL commands **make the transaction boundary explicit**: they decide whether the changes of a transaction become permanent or are undone. A transaction is a sequence of operations treated as one logical unit (atomicity); TCL is how the programmer declares where it starts and ends.

Command	Syntax	Effect
COMMIT	`COMMIT;`	Ends the transaction successfully and makes **all** changes since the last commit permanent; writes the commit record to the log and releases locks. Point of no return.
ROLLBACK	`ROLLBACK;`	Aborts the transaction and undoes **all** changes back to the last COMMIT/START; restores before-images from the undo log; releases locks.
ROLLBACK TO savepoint	`ROLLBACK TO sv2;`	Undoes only the work done after `sv2`. The savepoint itself remains valid in Oracle; later savepoints are overwritten. Transaction is still open.
SAVEPOINT	`SAVEPOINT sv1;`	Marks an intermediate point inside a transaction so partial rollback becomes possible.
SET TRANSACTION	`SET TRANSACTION NAME 'payroll';` / `READ ONLY` / `ISOLATION LEVEL SERIALIZABLE`	Names the transaction, or makes it read-consistent, or fixes its isolation level.
Implicit commit	DDL (`CREATE/ALTER/DROP`) auto-commits in Oracle; MySQL DDL too	Worth one line — DML needs explicit COMMIT in Oracle, while MySQL autocommits each statement unless `autocommit=0`.

```
SAVEPOINT before_delete;
DELETE FROM student WHERE cgpa < 4;      -- wrong condition!
ROLLBACK TO before_delete;               -- rows are back
UPDATE student SET cgpa = cgpa WHERE roll_no = 101;
COMMIT;                                  -- now permanent
```

Also mention the **transaction states** (active → partially committed → committed | failed → rolled back) and that durability of COMMIT is provided by write-ahead logging, while rollback is provided by the undo log. A `WHERE`-less `DELETE` followed by `ROLLBACK` **can** be undone in Oracle (undo, not redo) but the same is not true of `TRUNCATE`, which is DDL and commits itself — the classic comparison.

Q.4 (C) What is database recovery? Explain different types of it. [10]

Database recovery is the process of restoring the database from an inconsistent state to the last **consistent, committed state** after a failure, using redundant data (backups + logs). It is the durability half of ACID. The **log** (sequence of records written before the data itself — *Write-Ahead Logging*) holds for each operation: `<T_i, X, V_old, V_new, status>` and `<T_i, COMMIT>` / `<T_i, ABORT>`.

Types of failure handled 1. **Transaction/system (software) error** — logical error, bad input, deadlock or timeout victim; the transaction aborts → **transaction rollback** undoes only that transaction. 2. **System crash** — hardware/power/OS/DBMS bug loses **main memory** but not disk; uncommitted transactions must be undone and committed-but-unwritten ones redone → recovery at restart scanning the log. 3. **Media (disk) failure** — a storage device is destroyed, losing data **and** the log → restored from a **backup** + archived log (the only type needing copies kept off the failed device). 4. **Block/corruption and network failures**, and catastrophic/site failures → standby replication.

Recovery techniques / types

(a) Log-based recovery — undo and/or redo driven by log records, with two checkpoint styles: * **Immediate-modification (STEAL)** — changed blocks may be written to disk before commit; the log stores **before-images**, so **undo** is available. Requires **undo for uncommitted** and **redo for committed** transactions. Dirty-page table tracks what was written. * **Deferred-update (no-steal)** — modified pages stay in the buffer until commit; nothing reaches disk of an aborted transaction, so only **REDO** (repeat history) is

needed, never UNDO — the classic ARIES-free scheme. * **Non-steal non-lose**: log before-images for undo, after-images for redo.

(b) Checkpointing — periodically write all dirty buffers and log a `<chkpt>` record listing active transactions and their last log addresses. Recovery then only scans **back to the last checkpoint** instead of the whole log — this bounds restart time. Types: **fuzzy/checkpoint with steal**, **analysis → redo → undo** phases (the ARIES three-pass structure: analysis determines winners/losers, REDO repeats history to reach the failure point, UNDO rolls back losers to a consistent state, using **CLR** — Compensation Log Records — so undo itself is idempotent and restart-safe).

(c) Shadow paging — the pre-image technique: two page tables, `current` and `next`; only the new copy is modified, and the `current` pointer is switched at commit. No log needed for atomicity and crashes leave the old structure intact. Drawbacks: **fragmentation**, no concurrency benefit unless combined with copying, expensive pointer updates, recovery from media failure still needs backup — so it is used rarely (some file systems and WAL alternates).

(d) Mirror / replication-based recovery — keep an identical copy (RAID-1, database mirroring, Oracle Data Guard, always-running standby) so a failed unit is replaced online. Handles **media failure**, the case logs alone cannot.

(e) Backup + archive-log restore — for media failure: restore the last **full backup** (cold/offline, hot/online, incremental differential/cumulative), then **roll forward** the archived logs to the desired point in time (PITR). Tools: Oracle RMAN/Data Pump, MySQL `mysqldump` / `xtrabackup` + binlog.

Roll-forward vs roll-backward: *roll forward* (redo) re-applies committed changes from the log onto the restored backup to bring it up to the failure moment; *roll backward* (undo) removes effects of uncommitted transactions. Most real recovery does both: restore backup → roll forward → undo losers.

Write the closing paragraph: the goal is **atomicity + durability under every failure class**; the price is log overhead and longer restart; and high availability (RAC, clustering, replication) reduces outage while recovery protects correctness.

Q.4 (A — OR branch) Discuss Authentication and its techniques. [4]

Authentication verifies *the identity* of a user or process claiming access to the database before any privilege is exercised — the first gate of database security, followed by **authorization** (what the verified identity may do). In DBMS terms it happens at connection time (`CREATE USER … IDENTIFIED BY …`) and can be enforced at the OS, network or database layer.

Techniques 1. **Password / passphrase** (something you know) — the classic `IDENTIFIED BY`; must be salted and hashed (never stored in clear text), with expiry, complexity policy and lockout after failed attempts. 2. **Passcode / PIN** — short numeric secret, usually for a second factor. 3. **Token-based / two-factor (something you have)** — hardware key, smart card, security-key fob, OTP from an authenticator app or SMS; time-based TOTP or challenge-response. 4. **Certificate-based** — X.509 digital certificates over SSL/TLS; `CREATE USER … IDENTIFIED EXTERNALLY` or `AS 'cn=…'`; mutual authentication for applications and DB links. 5. **Biometric (something you are)** — fingerprint, iris, face, voice; used for high-security terminals and mobile clients. 6. **Kerberos / network authentication** — ticket-granting service so the DB trusts a validated KDC ticket; `OS authentication`: `IDENTIFIED EXTERNALLY` lets the operating system or directory validate the user once (single sign-on). 7. **LDAP / Active Directory / RADIUS integration and SSO / federated identity (OAuth2, SAML)** — one identity store across enterprise services, with PKI as the trust backbone. 8. **Radius/AAA servers and privilege-delegation** — centralized accounting of who logged in from where. Strong authentication combines the three factors (multi-factor authentication). Also mention: password verifiers are stored hashed in the data dictionary, failed-attempt auditing, and that authentication is the pre-condition for DAC/MAC/RBAC decisions.

Q.4 (B — OR branch) Explain any 6 numeric functions with syntax and example. [6]

Same table as 2024 Q.4(A) — for 6 marks write **six**, not four: `ABS(n)`, `CEIL(n)`, `FLOOR(n)`, `ROUND(n,d)`, `TRUNC(n,d)`, `MOD(m,n)`, and mention `POWER`, `SQRT`, `SIGN`, `EXP`, `LN`, `LOG(b,x)` as extras, each with a computed example, plus one SELECT applying them.

Q.4 (C — OR branch) What is cursor? Explain in detail with its types. [10]

A **cursor** is a named private **SQL work area / memory allocation** that the server creates to process a `SELECT` or `DML` statement. It holds the **result set (the active set)** plus a **pointer to the current row**, so the program can retrieve rows **one at a time** — something pure SQL, which works on whole sets, cannot do. Every SQL statement gets a cursor: the PL/SQL engine manages some, the programmer manages others.

Explicit cursor — five steps

```
DECLARE
   cur_stud  CURSOR IS SELECT roll_no, name FROM student WHERE dept_id = &d;
   rec_stud  cur_stud%ROWTYPE;
BEGIN
   OPEN cur_stud;                          -- 2. run the query, bind vars, build active set
   LOOP
      FETCH cur_stud INTO rec_stud;        -- 3. one row per iteration, pointer advances
      EXIT WHEN cur_stud%NOTFOUND;         -- 4. stop when no more rows
      DBMS_OUTPUT.PUT_LINE(rec_stud.roll_no || ' ' || rec_stud.name);
   END LOOP;
   CLOSE cur_stud;                         -- 5. release the private area
END;
/
```

1. **DECLARE** `CURSOR name [(params)] [RETURN type] IS query;`
2. **OPEN** — allocates the area, binds parameters/variables, executes the query, positions the pointer before the first row.
3. **FETCH … INTO** — copies the current row into variables and moves the pointer.
4. **EXIT WHEN cursor%NOTFOUND** — the loop terminator.
5. **CLOSE** — discards the active set; the cursor may be reopened.

Cursor attributes (used only after OPEN, and prefixed by the cursor name; `%ISOPEN` only for explicit cursors): `%FOUND` (last FETCH returned a row), `%NOTFOUND`, `%ROWCOUNT` (rows fetched so far), `%ISOPEN`.

Types of cursor

Created by	PL/SQL automatically for every `SELECT…INTO` and `INSERT/UPDATE/DELETE`	The programmer, with `DECLARE CURSOR`
Control	Fully managed: opened, executed, fetched, closed for you	Manual: OPEN/FETCH/CLOSE (or a FOR loop)
Multiple rows	**Raises** `TOO_MANY_ROWS` — it can return only one row	Designed for many rows
Attributes	Available only with the `SQL` prefix: `SQL%FOUND`, `SQL%ROWCOUNT`, `SQL%NOTFOUND` (no `%ISOPEN`, always closed after)	`cursor_name%FOUND` etc.
Speed/overhead	Faster, less code	More code, more memory

Refinements worth listing for full marks * **Parameterised cursor** — `CURSOR c (d NUMBER) IS … WHERE dept_id = d;` then `OPEN c(3);`. * **Cursor FOR loop** — the shortest correct form; implicit OPEN/FETCH/EXIT/CLOSE: `FOR r IN cur_stud LOOP … END LOOP;` * **`WHERE CURRENT OF`** — update/delete

exactly the row the cursor is on: `UPDATE student SET cgpa = 10 WHERE CURRENT OF cur_stud;` (requires `SELECT … FOR UPDATE`). * **`SELECT … FOR UPDATE [OF col] [NOWAIT]`** — locks the rows so they cannot change while the cursor works on them; pairs with `WHERE CURRENT OF`. * **Cursor variable / `REF CURSOR`** — a pointer to a query result that can be opened for different queries and passed between PL/SQL and the client (`SYS_REFCURSOR`), the basis of returning result sets from procedures. * **Attribute cursors / `BULK COLLECT … LIMIT n`** — fetch many rows into a collection in one context switch for performance, with `FORALL` for bulk DML. * **Open/hold cursors and `OPEN … FOR`** in dynamic SQL (`EXECUTE IMMEDIATE`). Cautions: an un-closed cursor leaks memory and holds locks; the query result is a snapshot, so re-reading is not automatic unless the cursor is reopened; `%ROWCOUNT` of an implicit cursor is reset after each statement.

Q.5 (A) Describe GRANT and REVOKE commands. [4]

Same as 2024 Q.2(A). DCL: `GRANT privileges ON object TO user [WITH GRANT OPTION]`, `REVOKE [GRANT OPTION FOR] privileges ON object FROM user [CASCADE]`, object vs system privileges, roles, and the revocation-dependency behaviour (RESTRICT is the default, CASCADE removes derived grants).

Q.5 (B) Construct a PL/SQL block which finds out sum of first 10 natural numbers. [6]

```
DECLARE
   i   NUMBER := 1;
   s   NUMBER := 0;
BEGIN
   WHILE i <= 10 LOOP             -- could be FOR i IN 1..10 LOOP
      s := s + i;
      i := i + 1;
   END LOOP;
   DBMS_OUTPUT.PUT_LINE('Sum of first 10 natural numbers = ' || s);
END;
/
```

Output: `Sum of first 10 natural numbers = 55`.

Two accepted alternatives, both worth knowing so you can pick the shortest:

```
-- FOR loop form
DECLARE s NUMBER := 0;
BEGIN
   FOR i IN 1..10 LOOP s := s + i; END LOOP;
   DBMS_OUTPUT.PUT_LINE(s);
END; /

-- pure-SQL form (one row, no loop)
SELECT SUM(level) AS total FROM dual CONNECT BY level <= 10;
```

Marks go to: correct declarations, loop header with the right bound, the accumulator `s := s + i`, the increment (in the WHILE form), the output, and the terminating `/`. Forgetting `i := i + 1` in the WHILE version causes an infinite loop — the commonest lost mark here.

Q.5 (C) Convert the following relation in 2NF if it is not in 2NF: R(A,B,C,D), F = {A→B, C→D}. [10]

Step 1 — candidate key. Compute closures: `A⁺ = {A,B}` (not all) · `C⁺ = {C,D}` (not all) · `AC⁺ = {A,B,C,D} = U`. Neither A nor C alone is a superkey, so **AC is the only candidate key**. **Prime attributes** = {A, C}. **Non-prime** = {B, D}. **Step 2 — is it in 1NF?** Yes: all attributes are atomic and a PK exists, so the relation is in 1NF and 2NF/3NF are the next tests. **Step 3 — test 2NF.** 2NF demands that no non-prime attribute be **partially** dependent on a proper subset of a candidate key. The candidate key is composite (AC), so partial dependencies are possible: * `A → B` — B depends on A, which is a **part** of the key AC → **partial dependency → violates**

2NF. * `C → D` — D depends on C, also a part of AC → **second partial dependency → violates 2NF**. There are no dependencies between two non-prime attributes, so nothing *transitive* exists — but both FDs breach 2NF, and for the same reason they also breach 3NF: neither determinant (A, C) is a superkey and neither dependent (B, D) is a prime attribute. Failing 2NF automatically means failing 3NF, so the answer to "is it in 2NF?" is **no**, and the decomposition below repairs both levels at once. **Step 4 — decompose.** Put each partial dependency in its own relation, and keep a relation holding the full key:

```
R1(A, B)        FD: A → B        key: A
R2(C, D)        FD: C → D        key: C
R3(A, C)        key: AC          (no non-prime attributes)
```

Step 5 — verify the properties of the decomposition. * **Dependency preserving**: `A→B` is enforced inside R1, `C→D` inside R2 — both recovered without a join. ✓ * **Lossless join**: the projections of R on {A,B}, {C,D}, {A,C} join back to R; R3 contains the whole candidate key AC, and A, C are keys of R1, R2 respectively, so every join is lossless. ✓ * **Normal forms reached**: in each new relation the determinant **is** a superkey, so the result is not only in **2NF and 3NF but in BCNF**. R3 is a pure relationship table between two keys — the natural schema for a many-to-many association. * **Anomalies removed**: B can be updated in exactly one row; a new (C,D) pair can be inserted without needing an A; deleting a C row no longer loses an A→B fact.

Draw the final diagram as three boxes with the arrows `A → B`, `C → D`, and the key box `A C` — this is the "convert" the question asks for.

Q.5 (A — OR branch) What is normalization? Explain advantages and disadvantages. [4]

Normalization = a formal, FD-driven decomposition process that organises a relation into successively higher normal forms (1NF → 2NF → 3NF → BCNF → 4NF → 5NF) to eliminate redundant data and the insert/update/delete anomalies. **Advantages** — removes redundancy and saves storage; eliminates the three anomalies; enforces integrity naturally through smaller keyed tables; yields a stable, well-understood, flexible logical design; easier to maintain and extend; better for frequent-write OLTP systems; documentation of dependencies. **Disadvantages** — many tables → joins needed to reconstruct one business fact, so **read/query performance drops** and the optimizer works harder; more foreign keys and constraint checks cost CPU; over-normalization (deep BCNF/5NF) rarely pays for itself; reports and data-warehouse queries become complex; denormalization is then reintroduced deliberately; and it ignores access patterns, so design must be revisited with the workload.

Q.5 (B — OR branch) What is index? Explain sparse and dense index. [6]

Same as 2024 Q.4(B). Add one sentence on the definition of an index as an auxiliary access structure that speeds retrieval at the cost of extra space and slower inserts, then reproduce the comparison table and the multi-level extension.

Q.5 (C — OR branch) Construct a B-Tree for (5,3,21,1,9,2,13,7,12,10,8,4,12) with order 4. [10]

Convention: order 4 = at most **4 children/pointers** per node = at most **3 keys**; split when a node would hold 4 keys; the middle key moves up. Note **12 appears twice** — as a search-key B-tree stores the values as given, the second 12 lands beside the first in the same leaf; if the marker treats keys as unique the shape is identical, only the last leaf shows a single `[12]`.

Insert	Overflow node	Median promoted
5, 3, 21	— (root leaf `[3,5,21]`, now full)	—
1	`[1,3,5,21]`	**5** → root `[5]`, leaves `[1,3]` `[21]`
9, 2, 13	absorbed → `[1,2,3]`, `[9,13,21]`	—
7	`[7,9,13,21]`	**13** → root `[5,13]`, leaves `[1,2,3]` `[7,9]` `[21]`
12	absorbed → `[7,9,12]`	—
10	`[7,9,10,12]`	**10** → root `[5,10,13]`, leaves `[1,2,3]` `[7,9]` `[12]` `[21]` (root now full)
8	absorbed → `[7,8,9]`	—
4	`[1,2,3,4]`	**3** → root would hold 4 keys, so the **root splits**: 10 becomes the new root
12 (dup)	goes to the `[12]` leaf	—

Final B-tree:

```
level 0 (root)        [ 10 ]
                     /      \
level 1          [ 3 | 5 ]   [ 13 ]
                 /    |   \     |   \
level 2 (leaves)[1,2] [4] [7,8,9] [12,12] [21]
```

Verification: height 2 for 13 keys; every internal node has 2–4 children; every non-root node holds 1–3 keys ($\lceil 4/2 \rceil - 1 = 1$ minimum) ✓; ranges are <3, 3–5, 5–10, 10–13, 13–21, ≥ 21 consistent with separators ✓; in-order traversal gives `1,2,3,4,5,7,8,9,10,12,12,13,21` ✓.

---

5. Questions from Assignments 1–4 that no past paper has used yet

These are the strongest candidates for *your* paper: the setter rotates the bank, and these have been assigned but not yet examined.

A1 Q3 — Explain Object-Oriented data models.

The **Object-Oriented Data Model (OODM)** represents data as **objects** that bundle state (attributes/instance variables) with behaviour (methods), directly modelling real-world entities — the answer to the limitations of relational and E-R models for complex/semantic data (CAD, multimedia, GIS, OODBMS: ObjectStore, Objectivity, OntoBase; Oracle's object-relational features). Core concepts: * **Object** — an entity with identity, state and behaviour; `oid` is immutable and system-generated, so two objects with identical values are still distinct (identity ≠ equality — the relational model has no such notion). * **Class** — template/abstraction of objects sharing attributes and methods; defines **extent** (all instances). * **Encapsulation / information hiding** — internal structure hidden, public interface only; supports DI naturally. * **Inheritance** — subclass reuses and extends superclass (single, multiple, hierarchical); supports code reuse and schema evolution. * **Polymorphism & dynamic binding** — the same message invokes different methods by type, resolved at run time. * **Methods and messages**, **overloading/overriding**. * **Complex/structured attributes** and **collection types** — set/bag/list/array of objects, so one attribute may hold a nested object (vs 1NF atomicity). * **Abstract class**, **instantiation**, **persistence**. Representation: `CLASS Employee(id, name, salary; method calc_pay())` with subclass `Manager inherits Employee adds bonus()`. **Advantages**: models rich semantics and complex data,

no impedance mismatch with OO languages (C++/Java), extensible types, better performance for navigative access. **Disadvantages**: no simple mathematical foundation, standards are weak (ODMG only), querying complex structures is hard, no mature declarative language comparable to SQL, ad-hoc querying/reporting weaker, DBMS complexity and cost. Related: **ORDBMS** blends both (Oracle, PostgreSQL) — object types, REFs, VARRAYs, nested tables, table inheritance.

A2 Q3 — What is dependency? Explain types of it.

A **dependency** is an integrity constraint / relationship between attributes of a relation that must hold in every legal instance; the formal basis of normalization. 1. **Functional dependency (FD)** `X → Y` — values of X determine a unique value of Y. Sub-types: **trivial** (Y ⊆ X), **non-trivial**, **fully functional dependency** (whole X needed), **partial dependency** (a proper subset of a candidate key determines a non-prime attribute — breaks 2NF), **transitive dependency** (`X→Y→Z` with Z non-prime — breaks 3NF). 2. **Multi-valued dependency (MVD)** `X ↠ Y` — for each X there is an independent, multi-valued set of Y; the complement Z is also multi-valued. Non-trivial MVD with X not a superkey breaks **4NF** (e.g. a professor teaching many subjects and authoring many books independently). 3. **Join dependency (JD)** $*(R_1,\ldots,R_n)$ — R equals the natural join of its projections; violated when only *joint* constraints exist, breaking **5NF**. 4. **Equality-generating / tuple-generating dependencies**, **inclusion dependency** (the basis of foreign keys: `R.A ⊆ S.B`), **domain-key dependency (6NF)**, and **embedded dependencies** (hold in some but not all relations).

A2 Q5 — R(A,B,C,D,E), F = {A→B, B→E, C→D}. Is it in 3NF? If not, decompose.

Closures: $A^+ = \{A,B,E\}$, $B^+ = \{B,E\}$, $C^+ = \{C,D\}$, $AC^+ = \{A,B,C,D,E\} = U$, and no single attribute or other pair covers U. So the **only candidate key is AC**; **prime = {A, C}**, **non-prime = {B, D, E}**. **2NF test** — `A → B`: A is a proper subset of the key AC and B is non-prime → **partial dependency, fails 2NF**. **3NF test** (for every FD `X→A`: X is a superkey **or** A is prime): * `A→B` — A not a superkey, B not prime → **violates 3NF**. * `B→E` — B not a superkey, E not prime → **violates 3NF** (and is a transitive dependency through A). * `C→D` — C not a superkey, D not prime → **violates 3NF**. **Conclusion: R is in 1NF only — neither 2NF nor 3NF.** **Decomposition (3NF synthesis algorithm)** — one relation per FD's attributes, then keep a relation containing a candidate key:

```
R1(A, B)   key A
R2(B, E)   key B
R3(C, D)   key C
R4(A, C)   key AC   ← preserves the candidate key, makes the join lossless
```

Each determinant is now a superkey of its own relation → **BCNF** level for R1–R3. The decomposition is **dependency preserving** (all three FDs enforced locally) and **lossless** (R4 carries the key AC, and A, C are keys of R1, R3, so joining recovers R exactly).

A2 Q7 — Write a short note on relational calculus.

Relational calculus is the **non-procedural/declarative** query language of Codd (1972): you state *what* result is required using predicates over variables, not *how* to compute it. Two forms: * **Tuple Relational Calculus (TRC)** — `{ t | P(t) }`, t is a **tuple variable** ranging over relations. "Names of CSE students": `{ t.name | Student(t) ∧ t.dept = 'CSE' }`. Universal `∀t(P(t))` is shorthand for `¬∃t(¬P(t))`. * **Domain Relational Calculus (DRC)** — $\{ \langle x_1,\ldots,x_n\rangle \mid P(x_1,\ldots,x_n) \}$, variables range over **domains (attribute values)**. `{ <r,n> | Student(r,n,d) ∧ d='CSE' }`. **Safe expression**: only finitely many tuples satisfy it — unlimited domains otherwise make a query uncomputable, so a DBMS must restrict to *range-limited* variables (range declarations `t ∈ Student`). SQL corresponds to the safe subset of TRC (each `SELECT–FROM–WHERE` is `{t | Student(t) ∧ …}`). **Expressive power**: TRC ≡ DRC ≡ relational algebra restricted to {σ, π, ×, ∪, −} (**Codd's theorem**). Algebra is procedural, calculus is declarative; algebra can count/order only with extensions, calculus

needs aggregates added. The **domain independent** requirement ensures the answer doesn't change when the underlying domains grow. Also known as the **calculus foundation of the "declarative" nature of SQL** — the optimizer translates a calculus-style query into an algebraic execution plan.

A3 Q4 — How does two-phase locking differ from timestamp-based protocol? / Explain timestamp-ordering.

Both are **conservative, conflict-serializable** concurrency-control schemes; one **blocks**, the other **aborts**.

Aspect		
Mechanism	Locks: shared (read) and exclusive (write) on data items	Logical timestamps order reads/writes; no locks
Serial order produced	Order in which locks are granted (**not known in advance**)	Fixed = ascending **TS(T)** order, known from the start
When conflict arises	Later requester **waits** (blocked)	The operation that would violate order → transaction is **rolled back**
Deadlock	**Possible** (circular wait); needs wait-die/wound-wait, timeout or detection graph	**Impossible** — nothing is held, so no circular wait
Concurrency/cascading	Strict/rigorous 2PL prevents cascading aborts; basic 2PL allows them	Cascading aborts unless combined with strict scheduling
Progress	Good when contention is low; waits consume time	Bad under heavy contention — repeated aborts, starvation
Extra cost	Lock table memory, overhead of acquiring/releasing	Storing R-ts/W-ts per item, restarts with new timestamps
Variants	Basic, **Conservative (preemptive/static)** 2PL, **Strict**, **Rigorous**, multiple granularity, predicate locks	Basic TO, **Thomas' write rule**, **Multiversion TO (MVTO)**, chained TO, conservative TO
Guarantee	Conflict-serializable (**2PL theorem**)	Conflict-serializable in TS order

2PL phases: growing (acquire only) → shrinking (release only); a lock released may never be reacquired — that rule is exactly what yields serializability. **Conservative 2PL** demands *all* locks upfront (no deadlock, but hard to know the item set). **Strict 2PL** holds exclusive locks until commit/abort → recoverable, no cascading; **Rigorous** holds all locks to commit.

A3 Q5 — Explain DAC and MAC.

Both are **authorization (access-control) models** for database security, differing in *who* decides and *how binding* the rules are.

DAC — Discretionary Access Control. The **owner** of an object discretionarily grants/revokes privileges to other users. Implemented with the **authorisation matrix** (rows = subjects, columns = objects, cells = privileges such as select/insert/update/delete/grant/reference) or **access-control lists** attached to objects. `GRANT … WITH GRANT OPTION` propagates rights and forms **grant graphs** with a root (DBA) plus `REVOKE … CASCADE` to prune them. Flexible, easy to use, the SQL standard model. *Weaknesses*: **Trojan-horse / cascading-transfer risk** — a grantee can hand the privilege to an untrusted user; no protection against inference or leakage; management overhead as rights spread.

MAC — Mandatory Access Control. Access is governed by **system-wide, non-discretionary rules** set by a security officer; users and objects each carry fixed labels — objects are **classified** (Top Secret > Secret > Confidential > Unclassified) and subjects hold a **clearance**. Owner cannot override. Two classic policies: * **Bell–LaPadula (confidentiality)** — *Simple Security Property*: **no read up** (subject reads only at or below its level);

-property*: **no write down; plus **strong tranquility** (labels don't change during a session). Prevents leakage of secrets to lower levels. * **Biba (integrity)** — mirror image: **no read down**, **no write up**, keeping trusted data from being corrupted by less-trusted sources. **Lattice-based MAC** generalises both to a partially ordered set of security classes with least upper bound/greatest lower bound, used for multi-level relational databases. *Weaknesses*: rigid, administration-intensive, unsuitable for commercial environments where data is not classified; still needs inference control. Compare: DAC = **user-driven, flexible, commercial** (GRANT/REVOKE, and RBAC is its scalable middle ground); MAC = **system-driven, strict, military/government**. **Role-Based Access Control (RBAC)** sits between them: privileges attach to *roles*, users to roles, with least privilege, separation of duties and role hierarchy — it is what large enterprises actually deploy.

A4 Q3 — Define set operations.

Set operations require **union compatibility** (same degree, corresponding attributes from the same domain) and follow SQL rules: duplicates eliminated (except `UNION ALL`), NULLs treated as equal for the purposes of set difference, result column names taken from the first query, and all operators are **left-associative** with UNION/UNION ALL/EXCEPT/INTERSECT at equal precedence. Spelling differs by vendor: Oracle and PostgreSQL use `MINUS`, the SQL standard and MySQL (from 8.0.31) use `EXCEPT`, and MySQL only gained `INTERSECT` in 8.0.31 as well.

```
π / SQL equivalents (columns must align positionally, not by name):
UNION        -- rows in either or both, duplicates removed
UNION ALL    -- keeps duplicates: faster, no de-dup sort
INTERSECT    -- rows in both
MINUS / EXCEPT   -- rows in first but not second
```

```
SELECT roll_no FROM enrolled_spring
UNION
SELECT roll_no FROM enrolled_autumn;      -- students in either term

SELECT roll_no FROM students
MINUS
SELECT roll_no FROM fees_paid;            -- defaulters
```

Rules: each component SELECT must have the same number of expressions of compatible types; `ORDER BY` goes once at the end and may reference only the first query's column names or positions; parentheses control grouping; `INTERSECT` binds tighter than `UNION` in standard SQL. Also mention **Cartesian product** as the set-operation cousin (comma-joined FROM with no WHERE).

A4 Q4 — Explain syntax of procedure and functions.

Procedure — a named PL/SQL subprogram that performs actions; invoked as a **statement**, may return 0..many values through `OUT` / `IN OUT` parameters, cannot be used inside a SQL expression.

```
CREATE [OR REPLACE] PROCEDURE proc_name (
    p_in    IN     NUMBER,
    p_out   OUT    VARCHAR2,
    p_inout IN OUT NUMBER DEFAULT 0 )
IS | AS
    -- local declarations
BEGIN
    ...
EXCEPTION
    WHEN OTHERS THEN ROLLBACK; RAISE;
END proc_name;
/

EXECUTE proc_name(10, :x);        -- SQL*Plus call
CALL proc_name(10, :x);           -- SQL call
BEGIN proc_name(10, :x); END;     -- PL/SQL call
```

Function — must return **exactly one** value via `RETURN`; invoked inside an expression; usable in SQL if it has no OUT parameters and does only queries (no DML/commit) — and with `DETERMINISTIC` the result may be cached.

```
CREATE [OR REPLACE] FUNCTION func_name (p_a NUMBER, p_b NUMBER)
RETURN NUMBER
[DETERMINISTIC] [RESULT_CACHE]
IS | AS
BEGIN
    RETURN p_a + p_b;
END func_name;
/

SELECT func_name(cgpa, 1) FROM student;   -- in SQL
v := func_name(2,3);                       -- in PL/SQL
```

Parameter modes: `IN` (default, by value, read-only), `OUT` (by reference, must be assigned), `IN OUT` (both). Passing methods by value/by reference. **Difference to state in one line**: a function *computes and returns one value* and is an expression; a procedure *performs work* and returns through parameters and is a statement. Related: **packages** group procedures/functions/variables/types with public/private sections, keeping namespace clean and loaded once in the SGA; **anonymous vs named blocks**.

A4 Q6 — Define primary key, foreign key and unique key with syntax and example.

Purpose	Uniquely identifies a row	Prevents duplicate values in a column	Enforces referential integrity to another table
NULL	Not allowed	**Allowed**; the standard, Oracle and MySQL treat NULLs as distinct so many NULL rows are permitted (SQL Server allows only one)	Allowed (unless NOT NULL)
Count per table	**Exactly one**	Many	Many
Index created	Automatic clustered/unique index	Automatic non-clustered unique index	No automatic index in Oracle/MySQL (create one manually)
Syntax (column level)	`roll_no INT PRIMARY KEY`	`email VARCHAR(50) UNIQUE`	`dept_id INT REFERENCES department(dept_id)`
Syntax (table level)	`CONSTRAINT pk_st PRIMARY KEY (roll_no)`	`CONSTRAINT uq_email UNIQUE (email)`	`CONSTRAINT fk_dept FOREIGN KEY (dept_id) REFERENCES department(dept_id) ON DELETE CASCADE`
Composite	`PRIMARY KEY (roll_no, course_id)`	`UNIQUE (first,last)`	`FOREIGN KEY (a,b) REFERENCES t(x,y)`
FK actions: `ON DELETE / ON UPDATE {CASCADE | SET NULL | SET DEFAULT | NO ACTION | RESTRICT}` . A **candidate key** is any minimal superkey; choosing one makes it the primary key and the rest become **alternate keys**; **unique key ≈ alternate key**.			

A1 Q4 / A4 Q7 — already covered

Specialization, generalization and aggregation are answered inside §3 (ER model, items 4–6) and MySQL/Oracle differences inside §4 (Q.3B). Math and string functions are the numeric/character function tables of §3 (Q.4A/§4 Q.4B-OR).

A3 Q7 — Construct B-Tree for (2,3,5,7,11,17,19,23,29,31), pointers per node = 4.

Order 4 → max 3 keys per node. Overflows occur at 7 (promote 5), 19 (promote 17) and 31 (promote 29):

```
level 0 (root)        [ 5 | 17 | 29 ]
                      /    |    |    \
level 1 (leaves) [2,3]  [7,11]  [19,23]  [31]
```

10 keys, height 1, every leaf holds 1–3 keys, ranges <5 / 5–17 / 17–29 / ≥29. In-order traversal returns the sorted input ✓.

6. Ten-question rapid-fire sheet (last-hour revision)

1. **3 levels of ANSI/SPARC architecture** — external, conceptual, internal; mapping between them gives data independence.
2. **DDL/DML/DCL/TCL** — 5 CREATE-family / 3 data verbs / GRANT-REVOKE / COMMIT-ROLLBACK-SAVEPOINT.
3. **Relational algebra symbols** — σ π ρ ∪ − ∩ × ⋈ ÷ γ; every result is a relation (closure property).
4. **SQL execution order** — FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT.
5. **Armstrong** — reflexive, augmentative, transitive (+ union, decomposition, pseudo-transitive); closure X^+ finds keys.
6. **Normal forms in one line each** — 1NF atomic · 2NF no partial · 3NF no transitive · BCNF every determinant is a key · 4NF no non-trivial MVD · 5NF no non-trivial JD.
7. **ACID ↔ subsystem** — Atomicity=undo log, Consistency=constraints, Isolation=locking/MVCC, Durability=redo log + checkpoint.
8. **Anomalies of concurrency** — lost update, dirty read, non-repeatable read, phantom; fixed by 2PL/TO/levels.
9. **2PL phases** — growing then shrinking; strict 2PL holds X-locks to commit; TO never deadlocks but aborts.
10. **Index choice** — dense for secondary/non-ordering keys, sparse for ordered primary files; B^+-tree for ranges, hash for equality.

7. If you only prepare 8 things, prepare these

Ranked by expected marks using the 2024/2025/assignment overlap analysis:

1. **B-Tree construction, order 4 and 5** — 10 marks, appeared in both papers *and* the assignment. §3 Q.5(C), §4 Q.5(C), §5 A3 Q7.
2. **Relational algebra — all 13 operators with examples** — 10 marks both years.
3. **ACID properties with the fund-transfer example** — 10 marks both years, verbatim.
4. **Normalization: definitions + convert a given relation to 2NF/3NF** — 10 marks both years; practise the closure→candidate-key→violating-FD→decompose→verify-lossless-and-dependency-preserving routine until it is automatic.
5. **Cursor with all five steps and `%FOUND`-family attributes** — 10 marks in 2025, in assignment 4.
6. **Database recovery types (log-based, checkpoint, shadow paging, rollback/roll-forward)** — 10 marks in 2025.
7. **ER model components + specialisation/generalisation/aggregation + mapping to tables** — 10 marks both years.
8. **The Q.1 20-mark short-answer bank in §2** — cheapest marks in the paper; both years' lists are fully answered there.

For the 4-mark slots learn: types of join, GRANT/REVOKE, data independence, network model, Armstrong, timestamp protocol, transactional control commands, authentication techniques, PL/SQL max-of-three. For the 6-mark slots: sparse vs dense index, data abstraction, views, MySQL vs Oracle, numeric and character functions.

Answers written against the syllabus units in the same folder: U1 Database system architecture · U2 Data models · U3 Relational query languages · U4 Relational database design · U5 Query processing and optimization · U6 Storage strategies · U7 Transaction processing · U8 Database security · U9 SQL concepts ·

U10 PL/SQL concepts. The unit notes match these answers; where your notes use a different example, keep your notes' wording — the marks are for the structure of the answer.